Neural Surrogate Modeling for Snake Robot Locomotion:

Faster Reinforcement Learning via Learned Cosserat Rod Dynamics

Qifan Wen
Electrical and Computer Engineering
The Ohio State University
wen.679@buckeyemail.osu.edu

Advisor: Prof. Abhishek Gupta gupta.706@osu.edu

Abstract

[Placeholder: Write abstract after all experimental results are available.]

Contents

Notation **4**

1 Introduction **4**

2 Related Work **4**

2.1 Neural Surrogates for ODE/PDE 4

2.2 RL Control for Snake and Soft Robots 5

3 Physics Simulation Backends **7**

3.1 Cosserat Rod Formulation 7

3.2 CPG Control 8

3.3 PyElastica Backend 9

3.3.1 Method of Lines and Explicit Integration 11

3.3.2 Staggered Grid Discretization 12

3.3.3 External Force Handling 14

3.4 DisMech Backend 14

3.4.1 System Formulation 15

3.4.2 DisMech Algorithm: Discrete Elastic Rods 16

3.4.3 Discrete Elastic Rod Discretization 17
3.4.4 Implicit Time Integration 17
3.5 Backend Comparison 18

4 Data Collection **20**
4.1 Elastica Data Collection Pipeline 20
4.2 Dataset Quality 21

5 Neural Surrogate Model **22**
5.1 State and Action Representation for Elastica 22
5.2 Per-Element CPG Phase Encoding 23
5.3 Surrogate Model Architecture 24
5.4 Configurations and Hyperparameters 25
5.4.1 Phase 1: Single-Step MSE (Epochs 1–20) 25
5.4.2 Phase 2: Combined Loss with Rollout (Epoch 20 Onward) 25
5.4.3 Density-Weighted Sampling 25
5.4.4 Hyperparameters 26
5.5 Periodic Pattern Learning 27
5.6 Results 27

6 Physics-Informed Surrogate **29**
6.1 Standard PINN 30
6.2 DD-PINN Method 30
6.3 Implementation 32
6.4 Results and Comparison 33

7 Reinforcement Learning **37**
7.1 Direct Simulation Training 37
7.1.1 Hyperparameters 37
7.1.2 Results 38
7.2 Surrogate Environment Training 40
7.2.1 Results 40
7.3 Challenges 41

8 Discussion **41**
8.1 Physics Calibration Challenges 41

8.2 Surrogate Architecture Experiments 42
8.3 Data Collection Pipeline Challenges 42
8.4 RL Training Challenges 43

9 Conclusion **44**

A Issue Tracker **44**
A.1 Physics Calibration Issues 44
A.2 Training Issues 45
A.3 TorchRL v0.11 Compatibility Issues 46
A.4 Performance and System Issues 47
A.5 Summary 47

Notation

Symbol	Dimension	Meaning
$\mathbf{s}_t$	$\mathbb{R}^{124}$	Rod state vector at time step t
$\mathbf{a}_t$	$[-1, 1]^5$	Normalized CPG action vector
T	$\mathbb{R}^{124} \times \mathbb{R}^5 \to \mathbb{R}^{124}$	Ground-truth transition operator (PyElastica)
f_θ	$\mathbb{R}^{189} \to \mathbb{R}^{124}$	Neural surrogate model with parameters θ
$\mathbf{z}_t$	$\mathbb{R}^{189}$	Surrogate input (state $\|$ action $\|$ phase encoding)
$\boldsymbol{\phi}_e$	$\mathbb{R}^3$	Per-element CPG phase triplet for element e
$\mathbf{x}(s,t)$	$\mathbb{R}^3$	Rod centerline position at arc length s
$\mathbf{d}(s,t)$	$\mathbb{R}^3$	Material director frame (orientation)
$\mathbf{F}$	$\mathbb{R}^3$	Internal force (shear and tension)
$\mathbf{M}$	$\mathbb{R}^3$	Internal moment (bending and torsion)
κ_e^{target}	$\mathbb{R}$	Target curvature at element e
$\mu_\Delta, \sigma_\Delta$	$\mathbb{R}^{124}$	Training-set mean and std of state deltas
N	21	Number of discretization nodes
N_e	20	Number of discretization elements

1 Introduction

Reinforcement learning for soft and continuum robots is bottlenecked by the cost of first-principles simulation. A snake robot modeled as a Cosserat rod requires resolving elastic body dynamics, anisotropic ground friction, and internally generated muscle moments at every simulation substep — PyElastica (Naughton et al., 2021) uses 500 Position Verlet substeps per control step, costing $\sim$46 ms per environment step. This report presents a neural surrogate $f_\theta \approx T$ that replaces the transition operator with a single GPU forward pass ($\sim$0.001 ms), targeting a four-to-five order-of-magnitude speedup for RL training.

The pipeline has three stages: (1) parallel data collection from 16 independent PyElastica workers (section 4), (2) training a feedforward MLP surrogate on state-delta prediction with per-element CPG phase encoding (section 5), and (3) PPO training in the surrogate environment with a direct Elastica baseline for comparison (section 7). Two physics backends are investigated — PyElastica (explicit symplectic) and DisMech (implicit backward Euler) — providing complementary perspectives on surrogate fidelity (section 3).

2 Related Work

2.1 Neural Surrogates for ODE/PDE

Table 1 summarizes neural surrogate methods for elastic and soft robot dynamics that are directly relevant to the Cosserat rod setting considered here.

Table 1: Neural surrogates for soft robot / Cosserat rod dynamics.

Reference	Method	Architecture	Speedup	Accuracy	Key contribution
DD-PINN (Stolzle et al., 2025)	Domain-decoupled PINN	Feedforward NN	44,000×	3 mm EE error (2.3%)	Time domain decoupled from network; closed-form gradients; MPC at 70 Hz on GPU
KNODE-Cosserat (Hsieh et al., 2024)	Hybrid physics + Neural ODE	Neural ODE residual	—	58.7% vs. physics-only	Coarse Cosserat backbone + learned correction; trained on PyElastica data
SoRoLEX (Berdica et al., 2024)	Learned simulator for RL	LSTM	∼10×	Task-space obs.	Full pipeline: sim data → surrogate → PPO; JAX GPU parallelization
PINN Soft Robot (Author et al., 2025)	Physics-informed NN	PINN	467×	—	MPC at 47 Hz; as little as 1 training dataset; articulated soft robot
This work	Data-driven MLP surrogate	MLP (3×512)	∼46,000× (proj.)	*[Placeholder: —]*	Per-element CPG phase encoding; two-phase training (single-step + rollout); full 124-dim state

The most closely related prior work is SoRoLEX, which shares an identical pipeline structure (collect trajectories from a high-fidelity simulator, train a neural dynamics model, run large-scale RL in the learned environment). The present work differs in architecture (MLP vs. LSTM), prediction target (full 124-dimensional Cosserat rod state vs. task-space observation), and the explicit use of a multi-step rollout loss to limit autoregressive error accumulation.

2.2 RL Control for Snake and Soft Robots

Table 2 consolidates RL approaches for snake and soft robot control, spanning kinematic, rigid-body, and continuum-mechanics platforms.

Table 2: RL methods for snake and soft robot control.

Reference	Robot	Actions	Algo.	Simulator	Key contribution
Shi et al. (2020)	3-link kinem.	Joint vel. (disc.)	DQN	Analytical	SE(2) fiber bundle reduces state 5D→3D
Liu et al. (2021)	4-link soft	CPG + option (6)	PPO	Custom	Multi-agent game for contact-aware nav.
Liu et al. (2022)	9-link wheeled	Gait offset (1)	PPO	MuJoCo	RL modulates undulation for path following
Liu et al. (2023)	4-link soft	CPG tonic (4)	PPO	MuJoCo	12-level curriculum + domain randomization
Zheng et al. (2022)	7-link rigid	CPG (5)	PPO	MuJoCo	Two-phase curriculum; series elastic act.
Jiang et al. (2024)	11-joint CO-BRA	Dual CPG (7)	DDPG	MuJoCo	A*→RL→CPG→PID hierarchy
Schaffer et al. (2024)	Biohybrid worm	Muscle act. (42)	PPO	PyElastica	Adaptive muscle force ceilings
Choi and Tong (2025)	Soft manip.	$\Delta\kappa$ (5)	SAC	DisMech	Implicit Euler for contact-rich tasks
Bing et al. (2019)	Soft snake	CPG (5)	PPO	PyElastica	CPG reduces exploration vs. per-joint
Hong et al. (2026)	Soft robot	Joint torques	PPO	FEM	FEM essential for sim-to-real fidelity
Janner et al. (2019)	Rigid artic.	Joint torques	SAC+M	MuJoCo	Model-based short-horizon planning
Sanchez-Gonzalez et al. (2020)	Particles	Forces	—	Custom	Graph network learns mesh-based physics
This work	Soft snake	CPG (5)	PPO	Elast./DM	Surrogate for 4–5 OOM speedup

Several common themes emerge from these studies. CPG-based action spaces dominate snake robot RL, reducing the exploration burden relative to per-joint torque control (Bing et al., 2019; Jiang et al., 2024; Liu et al., 2021, 2023; Zheng et al., 2022). Curriculum learning and domain randomization are standard techniques for bridging the sim-to-real gap (Liu et al., 2023; Zheng et al., 2022). All approaches share the same simulation cost bottleneck that motivates this work: MuJoCo-based environments require $\sim$1 ms per step, while Cosserat rod simulators such as PyElastica and DisMech cost 12–46 ms per step. The neural surrogate strategy developed here can be viewed as a model-based RL method where the learned model is the full transition operator rather than a latent dynamics model, enabling direct policy transfer evaluation between surrogate and real simulator and providing a drop-in replacement for any of these physics backends.

3 Physics Simulation Backends

3.1 Cosserat Rod Formulation

The Elastica backend solves a well-posed initial–boundary value problem for the Cosserat rod (Naughton et al., 2021; Till et al., 2019). The system is fully specified by the tuple $\mathcal{S} = (\mathcal{L},\ \Omega,\ \text{BC},\ \text{IC},\ \boldsymbol{\theta})$:

Table 3: Well-posedness tuple for the Elastica Cosserat rod system.

Component	Name	Specification
$\mathcal{L}$	Governing equations	Cosserat rod PDEs: conservation of linear and angular momentum (eqs. (1a) and (1b)) with linear elastic constitutive law (eq. (2))
Ω	Domain	$s \in [0, L]$, $L = 0.5\,\text{m}$ (arc length) $\times\ t \in [0, T]$ (time)
BC	Boundary conditions	Free–free: $\mathbf{F} = \mathbf{M} = \mathbf{0}$ at $s = 0$ and $s = L$
IC	Initial conditions	Straight rod at rest: $\mathbf{x}(s, 0) = \mathbf{x}_0 + s\hat{\mathbf{e}}_x,\ \mathbf{v} = \boldsymbol{\kappa} = \boldsymbol{\omega} = \mathbf{0}$
$\boldsymbol{\theta}$	Parameters	E=10^5 Pa, ν=0.5, ρ=1200 kg/m^3, R=0.02 m, c_t=0.01, c_n=0.05, γ=0.002
Control	CPG forcing	$\mathbf{a} = (A, f, k, \phi_0, b) \in \mathbb{R}^5$ enters as time-varying rest curvature $\boldsymbol{\kappa}_0(s, t)$

PDE (continuous).

$$\rho A\,\frac{\partial \mathbf{v}}{\partial t} = \frac{\partial \mathbf{F}}{\partial s} + \mathbf{f}_{\text{ext}} \qquad \text{(linear momentum)} \tag{1a}$$

$$\rho \mathbf{I}\,\frac{\partial \boldsymbol{\omega}}{\partial t} = \frac{\partial \mathbf{M}}{\partial s} + \mathbf{m}_{\text{ext}} + \frac{\partial \mathbf{x}}{\partial s} \times \mathbf{F} \qquad \text{(angular momentum)} \tag{1b}$$

Constitutive closure:

$$\mathbf{F} = \mathbf{S}\,(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_0), \quad \mathbf{M} = \mathbf{B}\,(\boldsymbol{\kappa} - \boldsymbol{\kappa}_0), \quad \mathbf{S} = \text{diag}(GA, GA, EA), \quad \mathbf{B} = \text{diag}(EI, EI, GJ) \tag{2}$$

ODE (semi-discrete). Spatial discretization on a staggered grid (N=21 nodes, N_e=20 elements) replaces the PDE with 124 coupled first-order ODEs:

$$\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s},\ \mathbf{a},\ t), \qquad \mathbf{s} \in \mathbb{R}^{124},\ \ \mathbf{a} \in \mathbb{R}^5 \tag{3}$$

Equation count per RL step. Each of the 124 state variables is advanced by one first-order ODE, giving 124 coupled equations. The Position Verlet integrator evaluates the right-hand side $\mathbf{g}(\mathbf{s}, \mathbf{a}, t)$ once per substep. With $n_{\text{sub}} = 500$ substeps per RL action, a

single RL step requires $500 \times 124 = 62{,}000$ scalar ODE evaluations, plus 500 curvature updates across 19 Voronoi nodes ($9{,}500$ trigonometric evaluations) and 500 friction force computations across 21 nodes ($10{,}500$ vector projections). The total per-step computational budget is approximately $82{,}000$ scalar operations, or $\sim$377,000 FLOPs including the matrix–vector products and cross products within each evaluation.

Integrator (fully discrete). Position Verlet ($\Delta t = 0.001$ s, $n_{\text{sub}} = 500$ substeps per RL step of $\Delta t_{\text{ctrl}} = 0.5$ s) advances the ODE to produce the transition operator:

$$\mathbf{s}_{t+1} = T(\mathbf{s}_t,\, \mathbf{a}_t) = \underbrace{V_{\Delta t}(\,\cdot\,;\, \boldsymbol{\kappa}^{\text{rest},499}) \circ \cdots \circ V_{\Delta t}(\,\cdot\,;\, \boldsymbol{\kappa}^{\text{rest},0})}_{\text{500 substeps, each with distinct rest curvature}}(\mathbf{s}_t) \tag{4}$$

Algorithm 1 expands this operator into its full computational form. The 2D restriction ($z \equiv 0$) is exact for planar locomotion.

Table 4: State vector $\mathbf{s}_t \in \mathbb{R}^{124}$ layout (Elastica).

Slice	Variable	Count	Meaning
`[0:21]`	x_i	21	Node x-positions (m)
`[21:42]`	y_i	21	Node y-positions (m)
`[42:63]`	$\dot{x}_i$	21	Node x-velocities (m s^{-1})
`[63:84]`	$\dot{y}_i$	21	Node y-velocities (m s^{-1})
`[84:104]`	ψ_e	20	Element yaw angle (rad)
`[104:124]`	$\omega_{z,e}$	20	Element angular velocity (rad s^{-1})

3.2 CPG Control

The Central Pattern Generator (CPG) drives locomotion by imposing a time-varying rest curvature on the rod. The RL agent selects five CPG wave parameters per control step:

Table 5: Action vector $\mathbf{a}_t \in \mathbb{R}^5$: CPG wave parameters.

Symbol	Name	Range	Units
A	Amplitude	$[0, 5]$	rad m^{-1}
f	Frequency	$[0.5, 3.0]$	Hz
k	Wave number	$[0.5, 3.5]$	wavelengths/body
ϕ_0	Phase offset	$[0, 2\pi]$	rad
b	Turn bias	$[-2, 2]$	rad m^{-1}

These parameters define the serpenoid rest curvature at each Voronoi node:

$$\kappa_v^{\text{rest}}(t) = A \sin(2\pi k \cdot s_v + 2\pi f \cdot t + \phi_0) + b, \qquad v = 1, \ldots, 19. \tag{5}$$

The action parameters are held fixed for one RL step ($\Delta t_{\text{ctrl}} = 0.5$ s), but κ_v^{rest} is recomputed at every substep because the temporal phase $2\pi f \cdot t$ advances by $\Delta\phi = 2\pi f \Delta t$ per substep.

Over one RL step, the phase sweeps through $2\pi f \cdot \Delta t_{\text{ctrl}}$ radians — up to 3π radians at $f = 3\,\text{Hz}$. The neural surrogate (section 5) replaces the entire transition operator T with a single forward pass, approximating the integrated effect of these 500 substeps.

DisMech applies curvature through the natural strain of discrete bend springs rather than the rest curvature array, and sets it once per RL step rather than per substep. The curvature is applied as:

$$\texttt{bend_springs.nat_strain}[j, 0] \leftarrow \kappa_j^{\text{rest}}, \quad j = 1, \ldots, 19. \tag{6}$$

Because the implicit solver takes a single $\Delta t = 0.05\,\text{s}$ step, the continuous curvature evolution that Elastica captures across 500 substeps is collapsed to a single snapshot at t_{step} in DisMech.

3.3 PyElastica Backend

The Elastica transition operator implements a Position Verlet integrator that advances the 124-dimensional ODE system through 500 substeps per RL action. Algorithm 1 expands this operator into its full computational form. The 2D restriction ($z \equiv 0$) is exact for planar locomotion.

Algorithm 1 Elastica Transition Operator $T(\mathbf{s}_t,\ \mathbf{a}_t,\ t_{\text{step}}) \to \mathbf{s}_{t+1}$

Ensure
Next state $\mathbf{s}_{t+1} \in \mathbb{R}^{124}$

Parameters
Δt=0.001 s – substep
n_{sub}=500 – substeps/step
Δt_{ctrl}=0.5 s – RL step
$\mathbf{S} \in \mathbb{R}^{3\times 3}$ – shear–stretch
$\mathbf{B} \in \mathbb{R}^{3\times 3}$ – bend–twist
$c_t, c_n \in \mathbb{R}$ – RFT friction
$\gamma \in \mathbb{R}$ – damping
$\mathbf{m} \in \mathbb{R}^{21}$ – nodal masses
$\mathbf{J} \in \mathbb{R}^{20}$ – elem inertias
$\mathbf{g} \in \mathbb{R}^2$ – gravity
$\odot, \oslash$ – elem-wise mult, div
$\mathbf{1} \in \mathbb{R}^2$ – ones (broadcast)

State
$\mathbf{q} \in \mathbb{R}^{62}$ – positions:
$x_i, y_i \in \mathbb{R}^{21\times 2}$
$\psi_e \in \mathbb{R}^{20}$
$\dot{\mathbf{q}} \in \mathbb{R}^{62}$ – velocities:
$\dot{x}_i, \dot{y}_i \in \mathbb{R}^{21\times 2}$
$\omega_{z,e} \in \mathbb{R}^{20}$
$\mathbf{s}$=$(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{124}$

CPG (table 5)
$A, f, k, \phi_0, b \in \mathbb{R}$ – from $\mathbf{a}_t$
$s_v \in [0, 1]$ – arc-length
$t^n \in \mathbb{R}$ – substep time

Grid ops (table 8)
$\mathbf{D} \in \mathbb{R}^{20\times 21}$ – node $\to$ elem
$\mathbf{D}^T \in \mathbb{R}^{21\times 20}$ – elem $\to$ node
$\mathbf{A}_v \in \mathbb{R}^{19\times 20}$ – elem $\to$ Vor.
$\mathbf{D}_v \in \mathbb{R}^{20\times 19}$ – Vor. $\to$ elem

Intermediates
$\boldsymbol{\kappa}^{\text{rest}} \in \mathbb{R}^{19}$ – target curv.
$\mathbf{q}^{1/2} \in \mathbb{R}^{62}$ – half-step state
$\mathbf{x}^{1/2} \in \mathbb{R}^{21\times 2}$ – half-step pos.
$\Delta\mathbf{x}_e \in \mathbb{R}^{20\times 2}$ – edge vectors
$\mathbf{t}_e \in \mathbb{R}^{20\times 2}$ – tangents
$\ell_e \in \mathbb{R}^{20}$ – lengths
$\ell_e^{\text{rest}} \in \mathbb{R}^{20}$ – rest lengths
$\varepsilon_e \in \mathbb{R}^{20}$ – dilatation
$\hat{\varepsilon}_v \in \mathbb{R}^{19}$ – Vor. dilatation
$\hat{\ell}_v \in \mathbb{R}^{19}$ – Vor. rest lengths
$\mathbf{Q}_e \in \mathbb{R}^{2\times 2}$ – rotation
$\boldsymbol{\sigma}_e \in \mathbb{R}^{20\times 2}$ – strain
$\boldsymbol{\sigma}_e^{\text{rest}} \in \mathbb{R}^{20\times 2}$ – rest strain
$\mathbf{n}_e^L \in \mathbb{R}^{20\times 2}$ – stress
$\boldsymbol{\kappa}_v \in \mathbb{R}^{19}$ – curvature
$\boldsymbol{\tau}_v^L \in \mathbb{R}^{19}$ – bend couple
$\dot{\mathbf{x}} \in \mathbb{R}^{21\times 2}$ – node velocities
$\mathbf{v}_t, \mathbf{v}_n \in \mathbb{R}^{21\times 2}$ – tang./norm.
$\mathbf{F}^{\text{int}} \in \mathbb{R}^{21\times 2}$ – int. forces
$\mathbf{F}^{\text{ext}} \in \mathbb{R}^{21\times 2}$ – ext. forces
$\mathbf{m}^{\text{int}} \in \mathbb{R}^{20}$ – int. torques
$\ddot{\mathbf{x}} \in \mathbb{R}^{21\times 2}$ – node accel.
$\dot{\boldsymbol{\omega}}_z \in \mathbb{R}^{20}$ – angular accel.

$\mathbf{s} \leftarrow \mathbf{s}_t$
for $n = 0, 1, \ldots, n_{\text{sub}}-1$ **do**
  // Control input: PDE forcing term $\boldsymbol{\kappa}_0(s, t)$
  $t^n \leftarrow t_{\text{step}} + n \cdot \Delta t$
  for $v = 1, \ldots, 19$ **do**
    $\kappa_v^{\text{rest}} \leftarrow A \sin(2\pi k \cdot s_v + 2\pi f \cdot t^n + \phi_0) + b$
  end for
  // Integrator: Verlet half-step
  $\mathbf{q}^{1/2} \leftarrow \mathbf{q} + \frac{1}{2}\Delta t\ \dot{\mathbf{q}}$
  // ODE right-hand side: discretized PDE
  // — geometry
  $\Delta\mathbf{x}_e \leftarrow \mathbf{D}\,\mathbf{x}^{1/2}$
  $\ell_e \leftarrow \|\Delta\mathbf{x}_e\|$
  $\mathbf{t}_e \leftarrow \Delta\mathbf{x}_e/\ell_e$
  $\varepsilon_e \leftarrow \ell_e/\ell_e^{\text{rest}}$
  // — shear–stretch: internal forces on nodes (translational)
  $\boldsymbol{\sigma}_e \leftarrow \mathbf{Q}_e^T(\Delta\mathbf{x}_e/\ell_e^{\text{rest}})$
  $\mathbf{n}_e^L \leftarrow \mathbf{S}\,(\boldsymbol{\sigma}_e - \boldsymbol{\sigma}_e^{\text{rest}})$
  $\mathbf{F}^{\text{int}} \leftarrow \mathbf{D}^T[\mathbf{Q}_e\,\mathbf{n}_e^L/\varepsilon_e]$
  // — bend–twist: internal torques on elements (rotational)
  $\boldsymbol{\kappa}_v \leftarrow$ curvature from directors at Voronoi
  $\boldsymbol{\tau}_v^L \leftarrow \mathbf{B}\,(\boldsymbol{\kappa}_v - \boldsymbol{\kappa}^{\text{rest}})$
  $\mathbf{m}^{\text{int}} \leftarrow \mathbf{D}_v[\boldsymbol{\tau}_v^L/\hat{\varepsilon}_v^3] + \mathbf{A}_v[(\boldsymbol{\kappa}_v \times \boldsymbol{\tau}_v^L)\hat{\ell}_v/\hat{\varepsilon}_v^3]$
    $+(\mathbf{Q}_e\mathbf{t}_e)\times\mathbf{n}_e^L\ell_e^{\text{rest}} + \ldots$
  // — external forces on nodes ($\mathbb{R}^{21\times 2}$)
  $\mathbf{F}^{\text{ext}} \leftarrow -c_t\mathbf{v}_t - c_n\mathbf{v}_n - \gamma\dot{\mathbf{x}} + \mathbf{m}\,\mathbf{g}^T$
  // — accelerations
  $\ddot{\mathbf{x}} \leftarrow (\mathbf{F}^{\text{int}} + \mathbf{F}^{\text{ext}}) \oslash \mathbf{m}\mathbf{1}^T$
  $\dot{\boldsymbol{\omega}}_z \leftarrow \mathbf{m}^{\text{int}} \oslash (\varepsilon_e \odot \mathbf{J})$
  // Integrator: Verlet velocity update + full-step
  $\dot{\mathbf{q}} \leftarrow \dot{\mathbf{q}} + \Delta t\ \ddot{\mathbf{q}}$
  $\mathbf{q} \leftarrow \mathbf{q}^{1/2} + \frac{1}{2}\Delta t\ \dot{\mathbf{q}}$
end for
return $\mathbf{s} = (\mathbf{q},\ \dot{\mathbf{q}})$

Cost: ∼754 scalar ops/substep × 500 = ∼377,000 ops/step (∼46 ms on CPU).
Key: $\boldsymbol{\kappa}^{\text{rest}}$ changes every substep — the CPG phase advances by $\Delta\phi = 2\pi f \Delta t$ per substep, accumulating $2\pi f \cdot \Delta t_{\text{ctrl}}$ radians over one RL step. The surrogate must learn the *integrated effect* of this traveling wave, not a single static curvature.

3.3.1 Method of Lines and Explicit Integration

The Cosserat rod is governed by PDEs in arc length s and time t, yet algorithm 1 contains no iterative solver — every line is a direct assignment. This is possible because of a **two-stage reduction** that converts the continuous PDE into a deterministic arithmetic map.

Stage 1 — spatial discretization (method of lines). All spatial derivatives $\partial/\partial s$ are replaced by sparse matrix–vector products on the staggered grid (section 3.3.2):

- Force gradient: $\partial \mathbf{F}/\partial s \;\rightarrow\; \mathbf{D}^T \mathbf{F}_e$ (bidiagonal, $O(N)$ ops).
- Strain from geometry: $\boldsymbol{\varepsilon}_e = \mathbf{Q}_e^T(\mathbf{x}_{e+1} - \mathbf{x}_e)/\ell_e^{\text{rest}}$ (subtract, rotate).
- Constitutive law: $\mathbf{F}_e = \mathbf{S}(\boldsymbol{\varepsilon}_e - \boldsymbol{\varepsilon}_e^{\text{rest}})$ (pointwise multiply).

After replacement, no spatial derivative remains. The PDE in (s, t) becomes the ODE system $\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s}, \mathbf{a}, t)$ (eq. (3)), where $\mathbf{g}$ is a **closed-form function** — a chain of geometry $\rightarrow$ strains $\rightarrow$ constitutive law $\rightarrow$ forces $\rightarrow$ accelerations, each computed by explicit algebra.

Stage 2 — explicit time integration. An explicit integrator advances the ODE by evaluating $\mathbf{g}$ at the *current* state and writing the result directly:

$$\mathbf{s}^{n+1} = \Phi(\mathbf{s}^n, \Delta t)$$

The unknown $\mathbf{s}^{n+1}$ appears *only on the left-hand side* — there is no equation to solve. Position Verlet evaluates $\mathbf{g}$ at a half-step configuration and updates positions and velocities in three sequential assignments. The result: each substep $V_{\Delta t}$ is a deterministic map $\mathbb{R}^{124} \rightarrow \mathbb{R}^{124}$ composed entirely of arithmetic.

Explicit vs. implicit integration. The defining distinction is whether the unknown state appears on the right-hand side.

Table 6: Explicit vs. implicit update rules.

Method	Update rule	Type
Forward Euler	$\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t\, \mathbf{g}(\mathbf{s}^n)$	Explicit
Backward Euler	$\mathbf{s}^{n+1} = \mathbf{s}^n + \Delta t\, \mathbf{g}(\mathbf{s}^{n+1})$	Implicit
Position Verlet (Elastica)	algorithm 1	Explicit

If $\mathbf{s}^{n+1}$ appears on the right-hand side, a Newton solve is required at every timestep (see section 3.4.4 for this approach in DisMech).

CFL stability constraint. Explicit methods are **conditionally stable**: the timestep must satisfy a Courant–Friedrichs–Lewy (CFL) bound set by the fastest wave mode (bending in

the stiffest element):

$$\Delta t < C\sqrt{\frac{\rho A\,\Delta s^4}{EI}}$$

With the project's rod parameters this gives $\Delta t_{\max} \approx 1.7 \times 10^{-3}$ s. PyElastica uses $\Delta t = 10^{-3}$ s, safely below the limit, but this forces $n_{\text{sub}} = 500$ substeps per RL action ($\Delta t_{\text{ctrl}} = 0.5$ s). Implicit methods are unconditionally stable — DisMech uses $\Delta t = 0.05$ s (50× larger) at the cost of a Newton solve per step. Table 14 provides the full property comparison between the two backends.

Surrogate implications. The neural surrogate replaces the 500-step composition $T(\mathbf{s}_t, \mathbf{a}_t) = V_{\Delta t}^{500}(\mathbf{s}_t; \mathbf{a}_t)$ (eq. (4)) — a **deterministic sequential map**, not a PDE solver. Each substep $V_{\Delta t}$ is a chain of explicit arithmetic (geometry, constitutive law, force balance, Verlet update); the computational cost arises from composing 500 such maps sequentially, with each depending on the output of the previous one. The surrogate collapses this sequential composition into a single forward pass. Because T is a smooth, deterministic function with no iterative convergence involved, a feedforward network can approximate the composed map directly — in contrast to DisMech (section 3.4), where the surrogate must additionally learn to bypass the Newton solver's convergence behavior.

3.3.2 Staggered Grid Discretization

The continuous Cosserat rod is discretized on a **staggered grid** with three interleaved layers (fig. 1). This structure explains the decomposition of the 124-dimensional state vector in table 4.

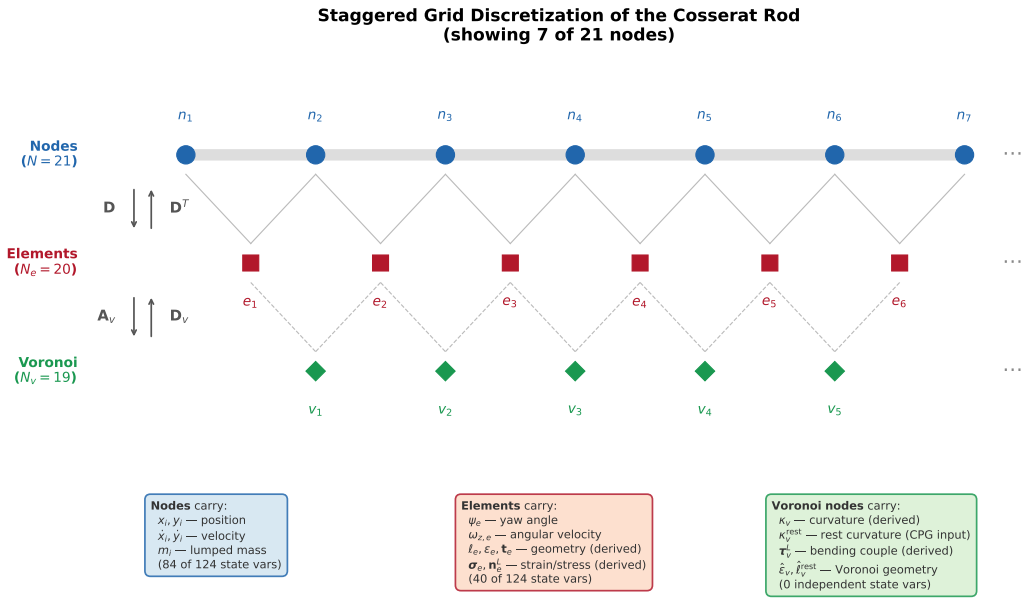

Figure 1: The three-layer staggered grid. Nodes (blue circles) sit at the rod vertices; elements (red squares) span between adjacent nodes; Voronoi nodes (green diamonds) sit between adjacent elements. Arrows show the sparse matrix operators that transfer information between layers.

Table 7: The three staggered-grid layers and their stored degrees of freedom.

Layer	Index	DOF	Variables	Total
Nodes	$i = 1, \ldots, 21$	Translational	$x_i, y_i, \dot{x}_i, \dot{y}_i$	84
Elements	$e = 1, \ldots, 20$	Rotational	$\psi_e, \omega_{z,e}$	40
Voronoi	$v = 1, \ldots, 19$	Derived	$\kappa_v, \kappa_v^{\text{rest}}$	0
Total independent state variables				**124**

Nodes are the rod vertices; elements are the segments connecting adjacent nodes; Voronoi nodes sit between adjacent elements and are the natural location for computing curvature κ_v — the quantity that couples the CPG control input to the rod's elastic response. Voronoi nodes carry no independent state; their quantities are derived from element values.

Four sparse matrix operators move information between layers:

Table 8: Sparse operators connecting the three staggered-grid layers. Each has $O(N)$ nonzero entries.

Operator	Direction	Size	Structure	Physical role
$\mathbf{D}$	Node $\rightarrow$ Element	20×21	Bidiagonal	Position differences $\rightarrow$ strain
$\mathbf{D}^T$	Element $\rightarrow$ Node	21×20	Bidiagonal	Internal stress $\rightarrow$ nodal forces
$\mathbf{A}_v$	Element $\rightarrow$ Voronoi	19×20	Tridiagonal	Element averages $\rightarrow$ curvature
$\mathbf{D}_v$	Voronoi $\rightarrow$ Element	20×19	Bidiagonal	Bending couple gradient $\rightarrow$ torques

3.3.3 External Force Handling

At each substep, the Elastica backend computes and applies the following external forces and torques to the rod. Table 9 summarizes all force types, their mathematical form, and where they act in the staggered grid.

Table 9: External forces and torques in PyElastica.

Force type	Acts on	Formula	Parameters	Notes
Gravity	Nodes (i)	$\mathbf{f}_i^{\text{grav}} = m_i \mathbf{g}$	$\mathbf{g} = (0, 0, -9.81)\,\text{m}\,\text{s}^{-2}$	Zero in 2D planar mode
RFT friction	Nodes (i)	$\mathbf{f}_i^{\text{rft}} = -c_t \mathbf{v}_{t,i} - c_n \mathbf{v}_{n,i}$	c_t=0.01, c_n=0.05	Anisotropic; enables serpentine propulsion
Numerical damping	Nodes (i)	$\mathbf{f}_i^{\text{damp}} = -\gamma \dot{\mathbf{x}}_i$	$\gamma = 0.002$	Suppresses high-freq. numerical ringing
Bending actuation	Voronoi (v)	$\boldsymbol{\kappa}_v^{\text{rest}}(t)$ via constitutive law	CPG parameters $\mathbf{a}_t$	Internal torque from elastic restoring force

The total external force at each node is $\mathbf{F}_i^{\text{ext}} = \mathbf{f}_i^{\text{grav}} + \mathbf{f}_i^{\text{rft}} + \mathbf{f}_i^{\text{damp}}$. RFT friction decomposes each node's velocity into tangential and normal components relative to the local rod tangent. The 5:1 anisotropy ratio ($c_n/c_t = 5$) produces net forward thrust during lateral undulation. PyElastica also supports Coulomb and Stribeck friction models, but RFT is used exclusively in this project for its computational simplicity and adequate fidelity for planar locomotion.

3.4 DisMech Backend

The second physics backend uses Discrete Elastic Rods (DER) via the DisMech simulator (Bergou et al., 2008, 2010). DisMech replaces PyElastica's explicit symplectic integration with implicit backward Euler, which is unconditionally stable and permits a single step of $\Delta t = 0.05\,\text{s}$ per RL action — eliminating the 500-substep inner loop. Algorithm 2 expands the resulting transition operator.

3.4.1 System Formulation

The Cosserat rod PDEs (eqs. (1a) and (1b)) and constitutive law (eq. (2)) apply identically. The well-posedness tuple $\mathcal{S}$ matches table 3 except for the solver parameters:

Table 10: Solver-parameter differences between backends (all other tuple entries are shared).

Parameter	PyElastica	DisMech
Integration scheme	Explicit symplectic (Verlet)	Implicit (backward Euler)
Timestep	$\Delta t_{\text{sub}} = 0.001\,\text{s}$	$\Delta t = 0.05\,\text{s}$
Steps per RL action	500 substeps	1 step
Stability	Conditional (CFL)	Unconditional
Damping	$\gamma = 0.002$	Numerical dissipation (implicit)
Solver tolerances	—	$\epsilon_f{=}10^{-4},\ \epsilon_d{=}10^{-2},$ max 25 iter

Primary unknowns. DER uses vertex positions and a material frame (Bishop frame). At each internal vertex i the discrete curvature binormal is:

$$\boldsymbol{\kappa}_i = \frac{2\,\mathbf{e}_{i-1} \times \mathbf{e}_i}{|\mathbf{e}_{i-1}|\,|\mathbf{e}_i| + \mathbf{e}_{i-1} \cdot \mathbf{e}_i}, \qquad \mathbf{e}_i = \mathbf{x}_{i+1} - \mathbf{x}_i. \tag{7}$$

Discrete bending energy.

$$E_{\text{bend},i} = \frac{1}{2\bar{\ell}_i}(\boldsymbol{\kappa}_i - \boldsymbol{\kappa}_i^{\text{rest}})^\top \mathbf{B}\,(\boldsymbol{\kappa}_i - \boldsymbol{\kappa}_i^{\text{rest}}), \tag{8}$$

where $\bar{\ell}_i$ is the Voronoi length (mean of adjacent edge lengths) and $\boldsymbol{\kappa}_i^{\text{rest}}$ is the rest curvature (control input).

State vector. The 124-dimensional layout matches table 4 exactly. Table 11 details how each component is extracted from DisMech's 3D state.

Table 11: State extraction: DisMech 3D → shared 124-dim representation.

Component	Source	Extraction	Count
x_i, y_i	$\mathbf{q} \in \mathbb{R}^{63}$	(x, y) of each node; $z \equiv 0$ (2D mode)	42
$\dot{x}_i, \dot{y}_i$	$\mathbf{u} \in \mathbb{R}^{63}$	(x, y) of each node velocity	42
ψ_e	Derived	atan2$(\Delta y_e,\ \Delta x_e)$ from segment tangents	20
$\omega_{z,e}$	Derived	$(\Delta x_e \Delta \dot{y}_e - \Delta y_e \Delta \dot{x}_e)/\lvert\Delta \mathbf{x}_e\rvert^2$	20
Total			**124**

3.4.2 DisMech Algorithm: Discrete Elastic Rods

The DisMech transition operator solves a single implicit backward Euler step per RL action via Newton iteration. Algorithm 2 expands the full computational procedure.

Algorithm 2 DisMech Transition Operator $T_{\text{DER}}(\mathbf{s}_t,\ \mathbf{a}_t,\ t_{\text{step}}) \to \mathbf{s}_{t+1}$

Ensure
Next state $\mathbf{s}_{t+1} \in \mathbb{R}^{124}$

Parameters
Δt=0.05 s – timestep
ϵ_f=10^{-4} – force tol.
ϵ_d=10^{-2} – disp. tol.
K=25 – max Newton iter
$\mathbf{S}$ – shear–stretch stiffness
$\mathbf{B}$ – bending stiffness
μ – friction coeff.
m_i – lumped mass at node i
$\mathbf{g}$ – gravity

State
$\mathbf{q} \in \mathbb{R}^{63}$ – positions:
x_i, y_i, z_i – node coords
($z_i \equiv 0$ in 2D mode)
$\mathbf{v} \in \mathbb{R}^{63}$ – velocities:
$\dot{x}_i, \dot{y}_i, \dot{z}_i$ – node vel.
$\mathbf{s}$=$(\mathbf{q}, \mathbf{v}) \in \mathbb{R}^{126}$

CPG (table 5)
$A, f, k, \phi_0, b \in \mathbb{R}$ – from $\mathbf{a}_t$
$s_j \in [0, 1]$ – arc-length
$t_{\text{step}} \in \mathbb{R}$ – RL time

Intermediates
$\boldsymbol{\kappa}^{\text{rest}} \in \mathbb{R}^{19}$ – target curv.
$\mathbf{q}^{(k)} \in \mathbb{R}^{63}$ – Newton iterate
$\mathbf{g} \in \mathbb{R}^{63}$ – residual
$\mathbf{J} \in \mathbb{R}^{63\times 63}$ – Jacobian (sparse, banded)
$\delta\mathbf{q} \in \mathbb{R}^{63}$ – correction
$\mathbf{e}_e \in \mathbb{R}^{20\times 2}$ – edge vectors
$\psi_e \in \mathbb{R}^{20}$ – yaw angles
$\omega_{z,e} \in \mathbb{R}^{20}$ – angular vel.

$\mathbf{s} \leftarrow \mathbf{s}_t$
  // Control input: set curvature once per RL step
$\omega \leftarrow 2\pi f$
for $j = 1, \ldots, 19$ **do**
  $\kappa_j^{\text{rest}} \leftarrow A \sin(2\pi k \cdot s_j + \omega \cdot t_{\text{step}} + \phi_0) + b$
end for
  // Apply to bend springs (DER control API)
for $j = 1, \ldots, 19$ **do**
  `nat_strain`$[j, 0] \leftarrow \kappa_j^{\text{rest}}$; `nat_strain`$[j, 1] \leftarrow 0$ ▷ Planar
end for
  // Initial guess: explicit Euler predictor
$\mathbf{q}^{(0)} \leftarrow \mathbf{q} + \Delta t\ \mathbf{v}$
  // Newton iterations: solve $\mathbf{g}(\mathbf{q}_{n+1}) = \mathbf{0}$ *(eq. (10))*
for $k = 0, 1, \ldots, K-1$ **do**
    // — assemble residual
  $\mathbf{F} \leftarrow \mathbf{F}_{\text{elastic}}(\mathbf{q}^{(k)}) + \mathbf{F}_{\text{ext}}(\mathbf{q}^{(k)})$
  $\mathbf{g} \leftarrow \mathbf{M}(\mathbf{q}^{(k)} - 2\mathbf{q} + \mathbf{q}_{\text{prev}}) - \Delta t^2\, \mathbf{F}$
    // — assemble Jacobian and solve
  $\mathbf{J} \leftarrow \partial\mathbf{g}/\partial\mathbf{q}\big|_{\mathbf{q}^{(k)}}$ ▷ Sparse banded
  $\delta\mathbf{q} \leftarrow \mathbf{J}^{-1}\mathbf{g}$ ▷ Direct factorization
  $\mathbf{q}^{(k+1)} \leftarrow \mathbf{q}^{(k)} - \delta\mathbf{q}$
    // — convergence check (table 13)
  if $\|\mathbf{g}\| < \epsilon_f$ **or** $\|\delta\mathbf{q}\| < \epsilon_d$ **then**
    break
  end if
end for
  // Update velocity from converged position
$\mathbf{v}' \leftarrow (\mathbf{q}^{(K)} - \mathbf{q})/\Delta t$
  // Pack 3D state → 2D 124-dim (table 11)
$(x_i, y_i) \leftarrow \mathbf{q}'_{3i:3i+2}$; $(\dot{x}_i, \dot{y}_i) \leftarrow \mathbf{v}'_{3i:3i+2}$ ▷ Drop z
$\mathbf{e}_e \leftarrow \mathbf{D}\,\mathbf{x}'$; $\psi_e \leftarrow \text{atan2}(e_{e,y},\ e_{e,x})$
$\omega_{z,e} \leftarrow (\Delta x_e \Delta\dot{y}_e - \Delta y_e \Delta\dot{x}_e)/|\Delta\mathbf{x}_e|^2$
return $\mathbf{s}_{t+1} = (x, y, \dot{x}, \dot{y}, \psi, \omega_z) \in \mathbb{R}^{124}$

Cost: 1 Newton solve with ∼3–5 iterations, ∼12 ms on CPU (sparse direct factorization). No substep loop.
Key: $\boldsymbol{\kappa}^{\text{rest}}$ is set *once* per RL step — the CPG wave is a single snapshot at t_{step}, unlike Elastica where the phase advances by $\Delta\phi = 2\pi f \Delta t_{\text{sub}}$ across 500 substeps. The surrogate must learn from this *coarser temporal resolution*.

3.4.3 Discrete Elastic Rod Discretization

DER follows a vertex–edge–hinge structure analogous to the staggered grid (fig. 1). Table 12 maps the DER terminology to the Elastica grid layers from table 7.

Table 12: DER discretization layers mapped to the staggered grid.

DER term	Elastica term	Index range	DOF type	Stores	Total
Vertices	Nodes	$i = 1, \ldots, 21$	Translational	$\mathbf{x}_i, \dot{\mathbf{x}}_i$	84
Edges	Elements	$e = 1, \ldots, 20$	Rotational	$\psi_e, \omega_{z,e}$	40
Hinges	Voronoi	$v = 1, \ldots, 19$	Derived	$\boldsymbol{\kappa}_v, \boldsymbol{\kappa}_v^{\text{rest}}$	0
Total independent state variables					**124**

The sparse operators $\mathbf{D}$, $\mathbf{D}^T$, $\mathbf{A}_v$, $\mathbf{D}_v$ (table 8) apply identically; DisMech implements them via direct array indexing rather than sparse matrix–vector products.

3.4.4 Implicit Time Integration

Backward Euler formulation. Given positions $\mathbf{q}_n$ and velocities $\mathbf{v}_n$:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t\, \mathbf{M}^{-1}\mathbf{F}(\mathbf{q}_{n+1},\, \mathbf{v}_{n+1}), \tag{9a}$$

$$\mathbf{q}_{n+1} = \mathbf{q}_n + \Delta t\, \mathbf{v}_{n+1}, \tag{9b}$$

where $\mathbf{M}$ is the mass matrix and $\mathbf{F}$ includes all internal elastic forces and external forces. $\mathbf{F}$ is evaluated at the unknown future state $(\mathbf{q}_{n+1}, \mathbf{v}_{n+1})$, making the system implicit.

Nonlinear residual. Substituting eq. (9b) into eq. (9a):

$$\mathbf{g}(\mathbf{q}_{n+1}) := \mathbf{M}\big(\mathbf{q}_{n+1} - 2\mathbf{q}_n + \mathbf{q}_{n-1}\big) - \Delta t^2\, \mathbf{F}(\mathbf{q}_{n+1}) = \mathbf{0}. \tag{10}$$

Newton solver. Each timestep solves eq. (10) iteratively:

$$\mathbf{q}^{(k+1)} = \mathbf{q}^{(k)} - \mathbf{J}^{-1}(\mathbf{q}^{(k)})\, \mathbf{g}(\mathbf{q}^{(k)}), \qquad \mathbf{J} = \frac{\partial \mathbf{g}}{\partial \mathbf{q}}. \tag{11}$$

Table 13: Newton solver convergence criteria.

Criterion	Condition	Default
Force residual	$\|\mathbf{g}(\mathbf{q}^{(k)})\| < \epsilon_f$	$\epsilon_f = 10^{-4}$
Displacement	$\|\delta\mathbf{q}\| < \epsilon_d$	$\epsilon_d = 10^{-2}$
Iteration limit	$k < K$	$K = 25$

Convergence is declared when either the force or displacement criterion is met. The Jacobian $\mathbf{J}$ (tangent stiffness + mass) is sparse; the linear system is solved via direct factorization. The full computational procedure is given in algorithm 2. The neural surrogate (section 5) replaces T_{DER} with a single forward pass, using the same architecture and training procedure as the Elastica surrogate. The only adaptation is the state extraction layer (table 11) that packs DisMech's 3D vertex state into the shared 124-dimensional representation.

3.5 Backend Comparison

Table 14 provides a comprehensive comparison of the two physics backends across solver properties, supported physics, and project metadata.

Table 14: Comprehensive comparison of PyElastica and DisMech.

Property	**PyElastica**	**DisMech**
Solver		
Physics model	Cosserat rod theory	Discrete Elastic Rods (DER)
Integration scheme	Explicit symplectic (Verlet / PEFRL)	Implicit backward Euler (Newton)
Timestep per call	$\Delta t_{\text{sub}} = 0.001\,\text{s}$	$\Delta t = 0.05\,\text{s}$
Steps per RL action	500 substeps	1 step
Stability	Conditional (CFL)	Unconditional
Per-step cost	$\sim$46 ms (500 explicit evals)	$\sim$12 ms (1 Newton solve)
Energy behavior	Bounded drift (symplectic)	Numerical dissipation
Linear system solve	None (explicit)	Sparse direct factorization
Supported forces and torques		
Shear–stretch	✓	✓
Bending moment	✓	✓
Torsion	✓	✓
Gravity	✓	✓
RFT (anisotropic drag)	✓	—
Coulomb / Stribeck friction	✓	—
Viscous damping	✓(γ=0.002)	✓(optional)
Self-contact	—	✓(FCL)
Control		
Curvature mechanism	Elastic rest shape (κ_0)	Hard constraint (natural strain)
CPG resolution	500 curvature updates/step	1 curvature snapshot/step
Project metadata		
Publication year	2021 (Naughton et al., 2021)	2008/2010 (Bergou et al., 2008, 2010)
Implementation language	Python (Numba JIT)	C++ (Python bindings)
License	MIT	MIT

Table 15 compares the internal and external force and torque types supported by each backend.

Table 15: Force and torque types: PyElastica vs. DisMech.

Force / Torque type	**PyElastica**	**DisMech**
Internal forces		
Shear–stretch (axial + transverse)	✓	✓
Bending moment (curvature)	✓	✓
Torsion (twist)	✓	✓
External forces		
Gravity	✓	✓
RFT (anisotropic velocity drag)	✓	—
Coulomb friction	✓	—
Stribeck friction	✓	—
Viscous damping (isotropic)	✓	✓
Self-contact (collision detection)	—	✓
Control torques		
Rest curvature (elastic target)	✓	—
Natural strain (hard constraint)	—	✓

The key difference in friction modeling is that DisMech's C++ backend does not support RFT at the native level; ground contact is modeled exclusively through viscous damping forces. PyElastica supports three friction models (RFT, Coulomb, Stribeck) but lacks self-contact detection, which DisMech provides via FCL-based collision.

The fundamental trade-off is between **temporal fidelity** and **computational cost**. PyElastica's 500 substeps capture the continuous evolution of the CPG curvature wave, but require $4\times$ more wall-clock time per RL step. DisMech's single implicit step is faster and unconditionally stable, but collapses the intra-step dynamics to a single curvature snapshot. For surrogate modeling, the Elastica backend presents a harder approximation target (the surrogate must learn the integrated effect of 500 distinct curvature configurations), while DisMech presents a simpler one-shot mapping.

4 Data Collection

4.1 Elastica Data Collection Pipeline

Training data for the surrogate model is generated by running 16 independent worker processes, each maintaining its own PyElastica Cosserat rod simulation instance. Actions are sampled from the 5-dimensional CPG action space using a Sobol quasi-random sequence, which distributes points more uniformly across all dimensions than pseudo-random sampling. To populate diverse regions of the state space, 30% of episodes begin with a state

perturbation: random offsets are applied to node positions, and angular velocity noise is injected into the rod elements with $\sigma_\omega = 1.5\,\mathrm{rad/s}$ (see section 8.3 for the rationale behind this value). Within each episode, 50% of actions are drawn uniformly at random and the remaining 50% follow the Sobol sequence.

Algorithm 3 Parallel data collection for surrogate training

Require: Number of workers $W = 16$, Sobol sequence $\mathcal{S}$, perturbation fraction $p = 0.3$
Ensure: Dataset $\mathcal{D} = \{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1})\}$

```
for each worker w = 1, ..., W in parallel do
    while not done do
        Initialize rod; with probability p, perturb state
        Sample action a from S (50%) or uniform (50%)
        Execute one RL step: s_{t+1} ← T(s_t, a_t)
        Store (s_t, a_t, s_{t+1}, t_start) to batch file
    end while
end for
```

The collection format stores one transition per row: the current state $\mathbf{s}_t$, the action $\mathbf{a}_t$, the next state $\mathbf{s}_{t+1}$, the episode start time t_{start}, and optionally the internal and external forces and torques acting on the rod. At approximately 2055 bytes per transition including forces, the target dataset size of 10 GB yields roughly 5 million transition pairs. A health monitor tracks the output rate of each worker and respawns any worker that stalls after initialization, ensuring sustained throughput across multi-hour collection runs.

Table 16: Data collection pipeline parameters.

Parameter	**Value**
Number of parallel workers	16
Action sampling	50% Sobol quasi-random, 50% uniform
State perturbation fraction	30% of episodes
Angular velocity perturbation σ_ω	1.5 rad/s
Transition size (with forces)	$\sim$2055 bytes
Target dataset size	$\sim$10 GB ($\sim$5M transitions)
Per-worker throughput	$\sim$22 transitions/s
Aggregate throughput	$\sim$350 transitions/s
Fields per transition	$\mathbf{s}_t$, $\mathbf{a}_t$, $\mathbf{s}_{t+1}$, t_{start}, forces (optional)

4.2 Dataset Quality

The final dataset contains 4,336,600 single-step transitions stored across 43,366 batch files. An automated validation pipeline checks the dataset against quality thresholds before surrogate training; the results are summarized in table 17.

Table 17: Data quality validation results.

Metric	Value	Threshold	Status
NaN/Inf rate	0.00%	$< 0.1\%$	Pass
Duplicate transitions	3.84% (166,518)	$< 1\%$	Fail
Constant features	0	0	Pass
Outlier rate ($> 5\sigma$)	0.096%	$< 0.5\%$	Pass
Action 5D fill fraction	74.5%	$> 5\%$	Pass
Per-dim action fill	100% (all dims)	$> 90\%$	Pass

Action coverage is excellent: Sobol quasi-random sampling achieves 100% per-dimension fill (20 bins) and 74.5% 5D joint fill across 3.2M bins, substantially better than pseudo-random sampling for the same budget. All five action dimensions (amplitude, frequency, wave number, phase offset, direction bias) have uniform marginal distributions (mean ≈ 0, std ≈ 0.577, consistent with $\mathcal{U}(-1, 1)$).

The 3.84% duplicate rate is attributed to the 1-step episode structure: workers that reset to identical initial conditions and receive the same Sobol action produce identical transitions. Deduplication yields 4,170,082 unique transitions. While the duplicate rate exceeds the automated threshold, these duplicates amount to implicit oversampling of common initial conditions and do not introduce label noise, so the dataset was used for training without deduplication.

Outlier analysis reveals that velocity ($\dot{x}$, $\dot{y}$) and angular velocity (ω_z) components account for 97% of all $> 5\sigma$ outliers. These arise from contact-induced velocity spikes during friction interactions and are physically plausible, so they were retained in the training set. The surrogate's difficulty predicting these components (section 5.6) is consistent with their high variance and heavy-tailed distributions in the training data.

5 Neural Surrogate Model

5.1 State and Action Representation for Elastica

The full 124-dimensional rod state vector $\mathbf{s}_t \in \mathbb{R}^{124}$, defined in table 4, serves as the Markov state for both surrogate training and RL. The 2D projection from the 3D Cosserat state is exact — z-components are identically zero — so no physical content is lost.

The action vector $\mathbf{a}_t \in [-1, 1]^5$ is linearly mapped to the physical CPG parameter ranges defined in table 5:

$$\mathbf{a}_{\text{phys}} = \frac{1}{2}\big[(\mathbf{a}_t + 1) \odot (\mathbf{a}_{\max} - \mathbf{a}_{\min}) + 2\,\mathbf{a}_{\min}\big] \tag{12}$$

- $\mathbf{a}_t \in [-1, 1]^5$ — normalized action from the RL policy
- $\mathbf{a}_{\max}$, $\mathbf{a}_{\min}$ — upper and lower bounds of each CPG parameter

- $\odot$ — element-wise multiplication

Denormalization to physical units occurs only inside the PyElastica environment; the surrogate receives $\mathbf{a}_t$ directly.

5.2 Per-Element CPG Phase Encoding

The target curvature at element e depends on both the global oscillation phase $2\pi ft$ and the element's arc-length position s_e through the serpenoid wave (algorithm 1, line 5). Elements at opposite ends of the rod can be at opposite phases of the curvature wave at the same instant. An early version of the surrogate (V1) used a global phase encoding $[\sin(2\pi ft), \cos(2\pi ft)] \in \mathbb{R}^2$ appended once to the input, requiring the model to reconstruct per-element phase offsets implicitly through shared MLP weights. This imposed a significant learning burden and contributed to poor angular velocity predictions ($R^2 = 0.23$).

The V2 encoding provides the per-element phase explicitly. For element $e \in \{1, \ldots, N_e\}$, an effective time is computed as:

$$t_e = \frac{k \cdot s_e}{f} \tag{13}$$

From this, a phase triplet is constructed:

$$\boldsymbol{\phi}_e = \left(\sin(2\pi f \cdot t_e),\ \cos(2\pi f \cdot t_e),\ \kappa_e^{\text{actual}}\right) \tag{14}$$

The effective time $t_e = k \cdot s_e / f$ incorporates the spatial phase offset at element e, where k is the wave number and s_e is the normalized arc-length position. The sine and cosine terms give the model the exact phase of the serpenoid wave at each element's location, and κ_e^{actual} is the instantaneous curvature providing the control error that drives the elastic restoring force.

Stacking $\boldsymbol{\phi}_e$ for all $N_e = 20$ elements yields a 60-dimensional phase feature vector. These features are computed on-the-fly during dataset loading from the stored action and episode start time t_{start}, so batch files contain only rod states, actions, and t_{start} without pre-computed phase data.

The full surrogate input concatenates the z-score normalized rod state $\bar{\mathbf{s}}_t$, the normalized action $\mathbf{a}_t$, and the per-element phase features:

$$\mathbf{z}_t = \left[\bar{\mathbf{s}}_t \parallel \mathbf{a}_t \parallel \boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_{N_e}\right] \in \mathbb{R}^{189} \tag{15}$$

- $\bar{\mathbf{s}}_t = (\mathbf{s}_t - \boldsymbol{\mu}_s)/\boldsymbol{\sigma}_s$ — z-score normalized state using training-set per-feature mean $\boldsymbol{\mu}_s$ and standard deviation $\boldsymbol{\sigma}_s$
- $\mathbf{a}_t \in [-1, 1]^5$ — normalized CPG action
- $\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_{N_e}$ — per-element phase triplets (60 dimensions total)

The 189 dimensions decompose as $124 + 5 + 60$.

5.3 Surrogate Model Architecture

The surrogate replaces the entire transition operator T with a single forward pass:

$$\underbrace{\text{Cosserat PDE}}_{\text{continuous}} \xrightarrow{\epsilon_{\text{space}}} \underbrace{\dot{\mathbf{s}} = \mathbf{g}(\mathbf{s}, \mathbf{a}, t)}_{\text{124-dim ODE}} \xrightarrow{\epsilon_{\text{time}}} \underbrace{T = V_{\Delta t}^{500}}_{\text{Alg. 1}} \xrightarrow{\epsilon_{\text{nn}}} \underbrace{f_\theta}_{\text{surrogate}} \tag{16}$$

$$f_\theta(\mathbf{z}_t) \;\approx\; T(\mathbf{s}_t,\, \mathbf{a}_t) - \mathbf{s}_t, \qquad \mathbf{z}_t \in \mathbb{R}^{189}, \;\; f_\theta \colon \mathbb{R}^{189} \to \mathbb{R}^{124} \tag{17}$$

The surrogate is a feedforward MLP. It maps the 189-dimensional input $\mathbf{z}_t$ to a 124-dimensional normalized state delta, from which the predicted next state is recovered through affine reconstruction:

$$f_\theta : \mathbf{z}_t \mapsto \overline{\Delta \mathbf{s}} \tag{18}$$

$$\hat{\mathbf{s}}_{t+1} = \mathbf{s}_t + \boldsymbol{\sigma}_\Delta \odot f_\theta(\mathbf{z}_t) + \boldsymbol{\mu}_\Delta \tag{19}$$

- f_θ — neural surrogate with parameters θ
- $\overline{\Delta \mathbf{s}}$ — predicted normalized state delta
- $\boldsymbol{\mu}_\Delta, \boldsymbol{\sigma}_\Delta \in \mathbb{R}^{124}$ — training-set mean and standard deviation of the state delta $(\mathbf{s}_{t+1} - \mathbf{s}_t)$, computed once before training begins

Delta prediction offers two advantages over predicting the absolute next state. First, per-step state changes are small relative to absolute values — position deltas are typically on the order of centimeters per control step while absolute positions span meters — so the target distribution is more concentrated and easier to fit. Second, when $f_\theta(\mathbf{z}_t) \approx \mathbf{0}$, the reconstruction yields $\hat{\mathbf{s}}_{t+1} \approx \mathbf{s}_t + \boldsymbol{\mu}_\Delta$, providing a reasonable default (the mean empirical transition) that acts as an implicit residual connection.

The network architecture is:

$$\mathbb{R}^{189} \xrightarrow{\text{Linear}} \big[\text{Linear}(512) \to \text{LayerNorm} \to \text{SiLU}\big]^{\times 3} \xrightarrow{\text{Linear}} \mathbb{R}^{124} \tag{20}$$

The output layer is initialized to zero so that the model starts by predicting zero delta. LayerNorm is applied after each linear transformation (before the SiLU activation) to stabilize training dynamics. The total number of trainable parameters is approximately 659K.

[Placeholder: Computational graph of the surrogate model showing input decomposition ($\bar{\mathbf{s}}_t$, $\mathbf{a}_t$, ϕ_e), the three-layer residual MLP, and the affine reconstruction $\hat{\mathbf{s}}_{t+1} = \mathbf{s}_t + \boldsymbol{\sigma}_\Delta \odot f_\theta(\mathbf{z}_t) + \boldsymbol{\mu}_\Delta$.]

Figure 2: Surrogate model computational graph.

5.4 Configurations and Hyperparameters

Training is organized in two phases that introduce the rollout loss gradually to avoid instability during early optimization.

5.4.1 Phase 1: Single-Step MSE (Epochs 1–20)

The model is trained to minimize the mean squared error on the normalized state delta:

$$\mathcal{L}_{\text{single}} = \frac{1}{N} \sum_{i=1}^{N} \left\| f_\theta(\mathbf{z}_t^{(i)}) - \overline{\Delta \mathbf{s}}_{\text{true}}^{(i)} \right\|^2 \tag{21}$$

The sum runs over N training samples, where $f_\theta(\mathbf{z}_t^{(i)})$ is the predicted normalized delta for sample i and $\overline{\Delta \mathbf{s}}_{\text{true}}^{(i)}$ is the corresponding ground truth. This phase establishes accurate one-step predictions before the more expensive rollout computation is introduced.

5.4.2 Phase 2: Combined Loss with Rollout (Epoch 20 Onward)

An 8-step autoregressive rollout loss is added to penalize error accumulation:

$$\mathcal{L} = \mathcal{L}_{\text{single}} + \lambda_r \cdot \mathcal{L}_{\text{rollout}} \tag{22}$$

$$\mathcal{L}_{\text{rollout}} = \frac{1}{L} \sum_{t=0}^{L-1} \left\| \hat{\mathbf{s}}_{t+1} - \mathbf{s}_{t+1}^{\text{true}} \right\|^2 \tag{23}$$

The rollout weight is $\lambda_r = 0.1$ and the rollout length is $L = 8$ steps. At each step, the model's own prediction $\hat{\mathbf{s}}_{t+1}$ is used as input for the next, and gradients are back-propagated through the full unrolled trajectory. This loss directly penalizes the drift between surrogate rollouts and ground-truth PyElastica trajectories — the primary failure mode when the surrogate is used as a substitute environment for RL training.

5.4.3 Density-Weighted Sampling

Sobol quasi-random action sampling produces a non-uniform distribution over the state space. To compensate, each training sample is weighted by the inverse of its estimated local density:

$$w^{(i)} = \frac{1}{\hat{p}(\mathbf{c}^{(i)})} \tag{24}$$

$$\mathbf{c}^{(i)} = \left[\bar{x}_{\text{CoM}},\ \bar{y}_{\text{CoM}},\ \|\dot{\mathbf{x}}\|,\ |\bar{\omega}_z| \right] \tag{25}$$

- $w^{(i)}$ — importance weight for sample i
- $\hat{p}$ — estimated local density (joint histogram with 20 bins per dimension)
- $\mathbf{c}^{(i)} \in \mathbb{R}^4$ — summary feature capturing center-of-mass position ($\bar{x}_{\text{CoM}}$, $\bar{y}_{\text{CoM}}$), speed

($\|\dot{\mathbf{x}}\|$), and mean angular velocity magnitude ($|\bar{\omega}_z|$)

Weights are clipped at $10\times$ the mean to prevent rare samples from dominating the gradient.

5.4.4 Hyperparameters

Training configurations are summarized in tables 18 and 19. All experiments were logged to Weights & Biases for tracking and comparison.

Table 18: Surrogate training hyperparameters (shared across all runs).

Hyperparameter	**Value**
Architecture	4-layer MLP (4×1024)
Activation	SiLU
Normalization	LayerNorm (pre-activation)
Output initialization	Zero
Prediction target	Residual (Δs)
Optimizer	Adam (β_1=0.9, β_2=0.999)
Learning rate	1×10^{-4} (cosine schedule, 5-epoch warmup)
Batch size	4096 (auto-scaled to $\sim$245K effective)
Mixed precision	bfloat16
Input/target normalization	Yes (per-feature)
Density weighting	Yes (20-bin histogram, $10\times$ clip)
State noise	$\sigma = 0.001$
Early stopping	Patience 30 (validation MSE)
Action dim	5
Total parameters	$\sim$3.43M

Table 19: Per-run configuration differences and results.

	Run A	**Run B (best)**	**Run C**
State dim	128	128	130
Dropout	0.0	0.1	0.1
Weight decay	10^{-5}	10^{-4}	10^{-4}
Epochs trained	80	110	57
Best val. loss	0.712	**0.601**	0.733
Overall R^2	—	**0.393**	0.258

5.5 Periodic Pattern Learning

The CPG's time-varying rest curvature creates a frequency-domain challenge for the surrogate. During each RL step, the serpenoid wave completes $n_{\text{cycles}} = f \cdot \Delta t_{\text{ctrl}}$ oscillation cycles, and the integrated dynamical response is an oscillatory function of the action parameters (f, k). Standard MLPs suffer from *spectral bias* — the tendency to learn low-frequency target components before high-frequency ones — making the $f \mapsto \Delta\omega_z$ relationship particularly difficult to capture.

The existing per-element phase encoding (section 5.2) addresses the *spatial-temporal phase* at each element. Three additional solutions target the remaining *frequency-domain* gap:

1. **Random Fourier Features on (f, k):** Map the normalized frequency and wave number through $\gamma(\mathbf{v}) = [\sin(2\pi\mathbf{B}\mathbf{v}),\ \cos(2\pi\mathbf{B}\mathbf{v})]$ with $\mathbf{B} \in \mathbb{R}^{32\times 2}$ sampled from $\mathcal{N}(0, \sigma^2)$. This provides sinusoidal basis functions that the MLP would otherwise need to synthesize internally, converting the hard nonlinear approximation into an easier regression.

2. **Explicit n_{cycles} encoding:** Two features $[\sin(2\pi n_{\text{cycles}}), \cos(2\pi n_{\text{cycles}})]$ directly encode how many oscillation cycles fit within one RL step.

3. **Harmonic phase features:** Extend the per-element encoding from the fundamental (H=1) to H=3 harmonics: $\sin(h\psi_j)$, $\cos(h\psi_j)$ for $h = 1, 2, 3$, capturing the rod's nonlinear response to sinusoidal forcing.

Alternative approaches — SIREN (periodic activations), Snake activations, Neural ODEs, and Fourier Neural Operators — were considered and rejected: the periodicity is in the *input encoding*, not the *network architecture*, and the surrogate's fixed-step, finite-dimensional structure does not match these approaches' strengths.

5.6 Results

The best surrogate (Run B) was trained for 110 epochs on 3.9M transitions (10% held out for validation), reaching a validation loss of 0.601 and an overall R^2 of 0.393. Figure 3 shows the training and validation loss curves alongside the overall R^2 evolution. The validation loss decreases steadily throughout training, with the train–validation gap widening after epoch 40, indicating mild overfitting. Adding dropout (p=0.1) and stronger weight decay (10^{-4}) in Run B reduced overfitting compared to Run A (val. loss 0.601 vs. 0.712).

Per-component accuracy. Prediction accuracy varies substantially across state components (table 20 and fig. 4). Center-of-mass position is predicted well ($R^2 = 0.852$), followed by relative element positions ($R^2 = 0.773$). These geometric quantities change smoothly and are directly constrained by the inextensibility of the rod. Angular velocity (ω_z, $R^2 = 0.320$) and heading (θ, $R^2 = 0.287$) are moderately predicted, while per-element yaw (ψ_e, $R^2 = 0.173$) and linear velocities ($\dot{x}, \dot{y}$, $R^2 = 0.145$) remain difficult. The velocity components exhibit the highest absolute MSE (0.081 and 0.213 for $\dot{x}$ and $\dot{y}$ respectively),

reflecting both their high variance in the training data and their sensitivity to contact and friction forces that are discontinuous across simulation substeps.

Figure 5 shows the per-component R^2 evolution during training. CoM position and relative positions converge rapidly (within 20 epochs), while angular and velocity components improve gradually and have not plateaued by epoch 110, suggesting that longer training or architectural changes could yield further gains.

Rollout loss. The two-phase training scheme described in section 5.4.1 was not activated in the best run: the rollout loss weight λ_r remained at zero, and all reported metrics reflect single-step prediction accuracy only. Multi-step rollout evaluation and the effect of the rollout loss on autoregressive error accumulation are deferred to future work alongside the architecture sweep (section 8.2).

Table 20: Surrogate single-step prediction accuracy by state component (Run B, epoch 110).

State component	R^2	MSE
CoM position $(\bar{x}, \bar{y})$	0.852	2.81×10^{-7}
Relative positions $(\Delta x_i, \Delta y_i)$	0.773	$\sim 3.2 \times 10^{-5}$
Angular velocity $(\omega_{z,e})$	0.320	1.442
Heading (θ)	0.287	2.49×10^{-3}
Yaw (ψ_e)	0.173	4.78×10^{-2}
Velocities $(\dot{x}_i, \dot{y}_i)$	0.145	8.10×10^{-2} / 2.13×10^{-1}
Overall	**0.393**	0.601

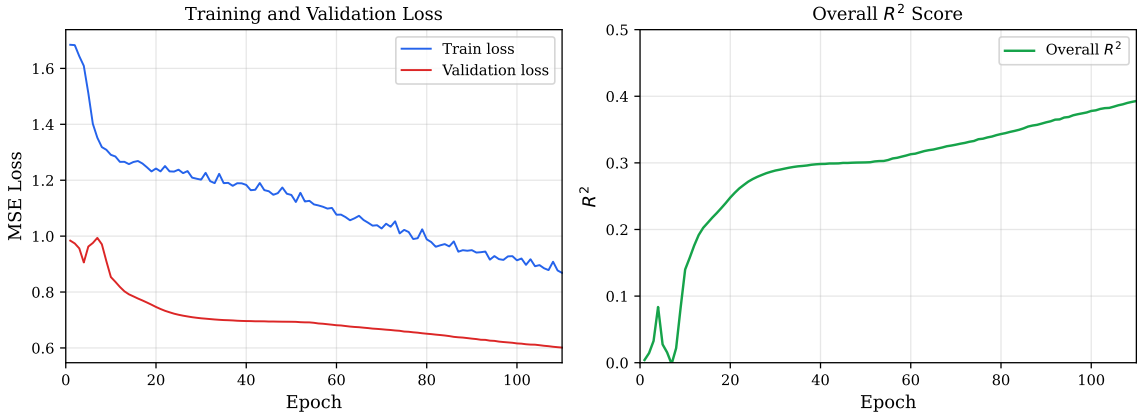

Figure 3: Training and validation MSE loss (left) and overall R^2 (right) over 110 epochs for the best surrogate run (Run B). The persistent train–validation gap indicates mild overfitting despite dropout and weight decay regularization.

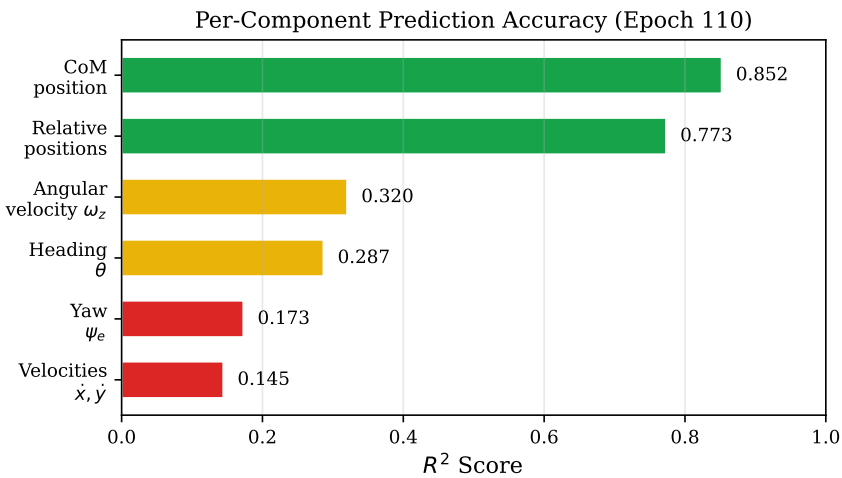

Figure 4: Per-component R^2 at epoch 110 (Run B). Geometric quantities (CoM, relative positions) are well predicted, while velocity-related components remain challenging.

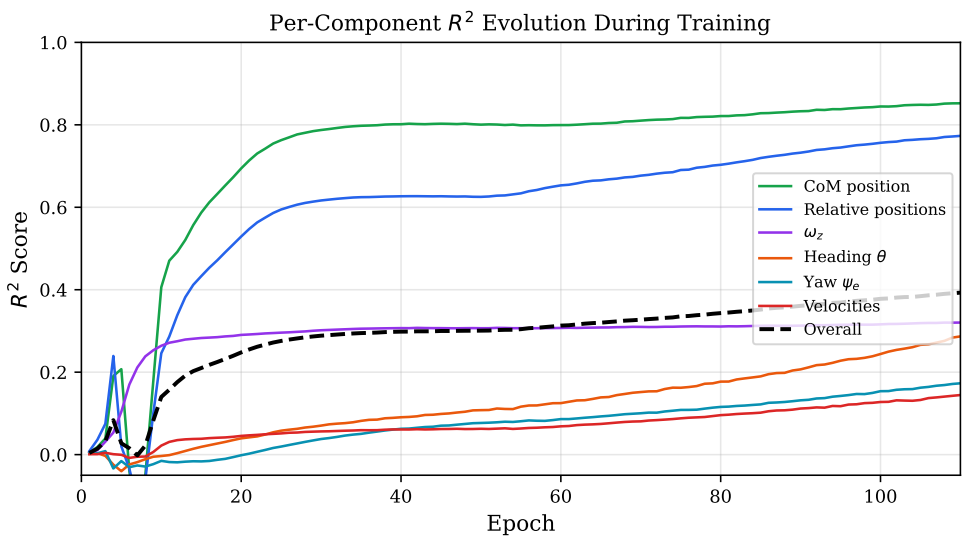

Figure 5: Per-component R^2 evolution during training. Position components converge within 20 epochs; angular and velocity components improve gradually and have not saturated by epoch 110.

6 Physics-Informed Surrogate

The data-driven surrogate of section 5 treats the simulator as a black box, learning state transitions purely from input–output examples. An alternative family of methods—*physics-informed neural networks* (PINNs)—embeds the known governing equations directly into the training loss, reducing data requirements and improving physical consistency of predictions. This section introduces the standard PINN formulation, its domain-decoupled variant (DD-PINN), the implementation developed in this project, and a comparative anal-

ysis.

6.1 Standard PINN

Physics-informed neural networks (Raissi et al., 2019) augment the purely data-driven loss with a *physics residual* term derived from the governing equations. Consider the state-space model

$$\dot{\mathbf{x}}(t) = \mathbf{f}_{\mathrm{SSM}}\big(\mathbf{x}(t),\, \mathbf{u}(t)\big), \qquad \mathbf{x}(0) = \mathbf{x}_0, \tag{26}$$

where $\mathbf{x}(t) \in \mathbb{R}^m$ is the state, $\mathbf{u}(t) \in \mathbb{R}^p$ is the input, and $\mathbf{f}_{\mathrm{SSM}}$ encodes the known (but expensive) physics—for the snake robot, the Cosserat rod equations of section 3.1. A feed-forward network $\mathbf{h}_\theta$ predicts the state at time t:

$$\hat{\mathbf{x}}_t = \mathbf{h}_\theta(\mathbf{x}_0,\, \mathbf{u},\, t). \tag{27}$$

The PINN loss combines three objectives:

$$\mathcal{L}(\theta) = \underbrace{\lambda_{\mathrm{IC}}\big\|\hat{\mathbf{x}}_0 - \mathbf{x}_0\big\|^2}_{\mathcal{L}_{\mathrm{IC}}} + \underbrace{\lambda_{\mathrm{phys}}\frac{1}{N_c}\sum_{k=1}^{N_c}\left\|\frac{\partial \hat{\mathbf{x}}_{t_k}}{\partial t} - \mathbf{f}_{\mathrm{SSM}}(\hat{\mathbf{x}}_{t_k},\, \mathbf{u}_{t_k})\right\|^2}_{\mathcal{L}_{\mathrm{phys}}} + \underbrace{\lambda_{\mathrm{data}}\frac{1}{N_d}\sum_{k=1}^{N_d}\|\hat{\mathbf{x}}_{t_k} - \mathbf{x}_{\mathrm{data},k}\|^2}_{\mathcal{L}_{\mathrm{data}}}. \tag{28}$$

The physics residual is evaluated at N_c *collocation points*—randomly sampled time instants where the governing equations are enforced without requiring simulation data. The time derivative $\partial\hat{\mathbf{x}}_t/\partial t$ is obtained via *automatic differentiation* (autodiff) through the network with respect to the input t.

While PINNs can train with little or no observed data, three practical bottlenecks limit their scalability:

1. **Expensive autodiff.** Computing $\partial\hat{\mathbf{x}}_t/\partial t$ requires an additional backward pass through the entire computational graph for each collocation point, adding significant cost for large state spaces.

2. **Loss balancing.** The three loss terms ($\mathcal{L}_{\mathrm{IC}}$, $\mathcal{L}_{\mathrm{phys}}$, $\mathcal{L}_{\mathrm{data}}$) compete during training and require careful weighting schemes.

3. **Training instability.** For systems with many coupled states ($m \gg 1$), PINN predictions can diverge.

6.2 DD-PINN Method

The *domain-decoupled* PINN (DD-PINN) (Krauss et al., 2024; Licher et al., 2025) resolves all three bottlenecks through a single architectural change: **remove time as a network input** and instead let the network output the parameters of an analytically differentiable *ansatz function*.

Core idea. The predicted state is decomposed as

$$\boxed{\hat{\mathbf{x}}_t = \mathbf{g}(\mathbf{a}_{\text{ans}}, t) + \mathbf{x}_0, \qquad \mathbf{a}_{\text{ans}} = \mathbf{f}_{\text{NN}}(\mathbf{x}_0, \mathbf{u}_0;\, \theta),} \tag{29}$$

where $\mathbf{f}_{\text{NN}}$ is a feedforward network and $\mathbf{g}$ is the ansatz. The ansatz must satisfy two properties: (i) differentiable in closed form for all $t \in [0, T_s]$, so $\dot{\mathbf{g}}$ can be computed analytically; and (ii) $\mathbf{g}(\mathbf{a}_{\text{ans}}, 0) \equiv \mathbf{0}$, which guarantees $\hat{\mathbf{x}}_0 = \mathbf{x}_0$ exactly.

Ansatz function. For each state variable $j \in \{1, \ldots, m\}$, the ansatz is a sum of n_g damped sinusoidal basis functions:

$$g_j(t) = \sum_{i=1}^{n_g} \alpha_{ij}\, e^{-\delta_{ij} t} \big[\phi_g(\beta_{ij}\, t + \gamma_{ij}) - \phi_g(\gamma_{ij})\big], \tag{30}$$

where $\phi_g(x) = \sin(x)$ and the subtraction $\phi_g(\gamma_{ij})$ ensures $g_j(0) = 0$. The network outputs the parameter vector $\mathbf{a}_{\text{ans}} = (\boldsymbol{\alpha},\, \boldsymbol{\beta},\, \boldsymbol{\gamma},\, \boldsymbol{\delta}) \in \mathbb{R}^{4mn_g}$, with each parameter playing an interpretable role: α_{ij} (amplitude), β_{ij} (frequency), γ_{ij} (phase offset), $\delta_{ij} \geq 0$ (damping rate).

Closed-form gradients. The time derivative is computed analytically:

$$\dot{g}_j(t) = \sum_{i=1}^{n_g} \alpha_{ij}\, e^{-\delta_{ij} t} \big[\beta_{ij}\, \cos(\beta_{ij}\, t + \gamma_{ij}) - \delta_{ij}\, \sin(\beta_{ij}\, t + \gamma_{ij})\big]. \tag{31}$$

These are elementary operations—no computational graph is constructed, and the cost scales linearly with $n_g \cdot m$ regardless of network depth.

Simplified loss. Since $\mathcal{L}_{\text{IC}} \equiv 0$ by construction, the training loss reduces to two terms:

$$\mathcal{L}(\theta) = \lambda_{\text{phys}}\, \mathcal{L}_{\text{phys}} + \lambda_{\text{data}}\, \mathcal{L}_{\text{data}}, \tag{32}$$

where $\mathcal{L}_{\text{phys}}$ and $\mathcal{L}_{\text{data}}$ follow the same form as in (28) but with $\dot{\mathbf{g}}(\mathbf{a}_{\text{ans}}, t_k)$ replacing the autodiff-based derivative. Because gradient evaluation is cheap, DD-PINN uses 10–25$\times$ more collocation points than standard PINNs, improving physics enforcement.

Inference pipeline. Figure 6 illustrates the DD-PINN inference flow. The neural network is evaluated *once* per prediction to produce ansatz parameters; the ansatz is then evaluated at arbitrarily many time points without additional network calls.

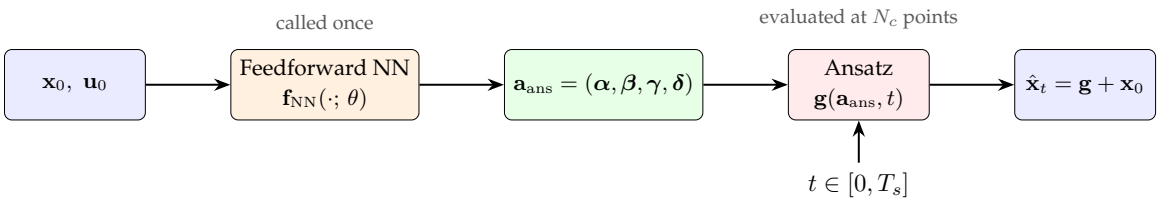

Figure 6: DD-PINN inference pipeline. The neural network is evaluated once per prediction to produce ansatz parameters $\mathbf{a}_{\mathrm{ans}}$. The ansatz $\mathbf{g}(\mathbf{a}_{\mathrm{ans}}, t)$ is then evaluated at arbitrarily many time points without additional network calls.

6.3 Implementation

Three physics-informed approaches were implemented in `src/pinn/`:

1. Physics-regularized surrogate. The simplest approach adds Cosserat rod physics constraints as soft regularizers to the existing MLP surrogate of section 5. The `PhysicsRegularizer` module computes four algebraic constraint losses: kinematic consistency in x and y (predicted displacement matches velocity-integrated displacement via the trapezoidal rule), angular consistency (yaw change matches integrated angular velocity), curvature–moment consistency (bending moment proportional to curvature), and energy conservation (kinetic energy change bounded). These constraints are weighted adaptively using *ReLoBRaLo* (Random Lookback with ReLU Balancing), which adjusts per-term weights via softmax over relative loss changes with exponential moving average smoothing. A lambda sweep over $\{0.001, 0.01, 0.1, 1.0\}$ was conducted to determine the optimal regularization strength.

2. Differentiable Cosserat RHS. For the full PINN and DD-PINN approaches, a differentiable `CosseratRHS` module computes the time derivative $\dot{\mathbf{x}} = \mathbf{f}_{\mathrm{SSM}}(\mathbf{x})$ for the 124-dimensional rod state, extending prior PINN work on static Cosserat rod approximation (Bensch et al., 2024) to the dynamic case. The module implements elastic bending moments via discrete curvature and full anisotropic resistive force theory (RFT) friction following the same formulation as the PyElastica backend (section 3.3). An inextensible rod approximation is used: stretching stiffness is omitted since bending dominates for planar snake locomotion. Gradient stability near zero element length is ensured by regularized tangent normalization $\hat{\mathbf{t}} = \Delta\mathbf{r}/\sqrt{\|\Delta\mathbf{r}\|^2 + \varepsilon^2}$, which provides smooth gradients where the standard normalization would be singular.

3. DD-PINN ansatz and model. The `DampedSinusoidalAnsatz` implements eq. (30) with softplus-enforced positive damping coefficients and exact initial condition satisfaction by construction. The `DDPINNModel` wraps a feedforward network with the ansatz and exposes the same `forward(state, action, time_encoding)` interface as the data-driven `SurrogateModel`, enabling drop-in substitution. A `FourierFeatureEmbedding` layer mitigates spectral bias by projecting inputs through random Fourier features before

the hidden layers. The output layer is zero-initialized so that the initial prediction is zero displacement (i.e., $\hat{\mathbf{x}}_t = \mathbf{x}_0$).

Collocation and nondimensionalization. Temporal collocation points are sampled using Sobol quasi-random sequences for uniform coverage of the prediction interval, with residual-based adaptive refinement (RAR) concentrating additional points where the physics residual is largest. All quantities are nondimensionalized using physics-based reference scales ($L_{\text{ref}} = 1.0\,\text{m}$, $t_{\text{ref}} = 0.5\,\text{s}$, $F_{\text{ref}} = EI/L^2$) to balance the magnitudes of position, velocity, and force terms in the residual loss.

6.4 Results and Comparison

Figures 7 to 9 present the PINN and DD-PINN results alongside the data-driven surrogate baseline.

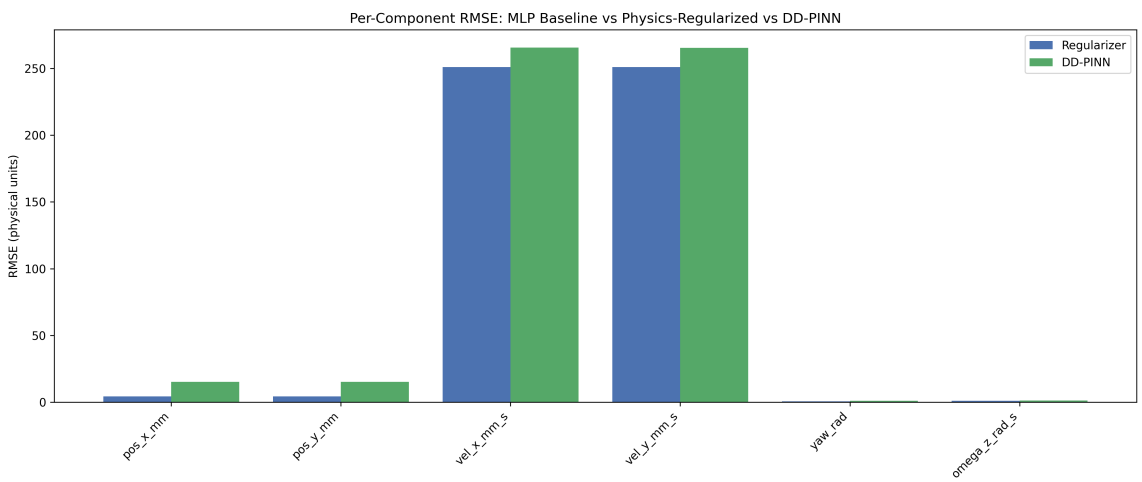

Figure 7: Per-component RMSE comparison across surrogate approaches. **[TODO: Fill in wall-clock training times after full runs.]**

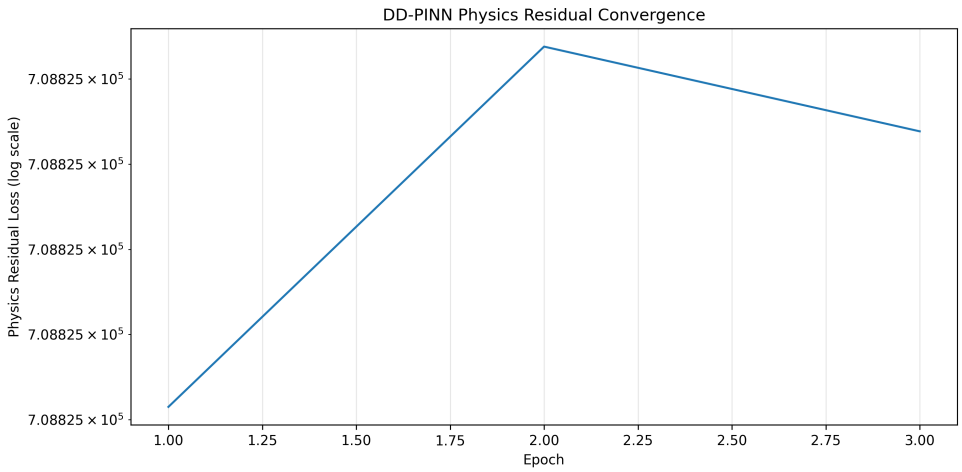

Figure 8: Physics residual loss convergence during training. The DD-PINN converges faster than the standard PINN due to cheap closed-form gradient evaluation enabling more collocation points per epoch.

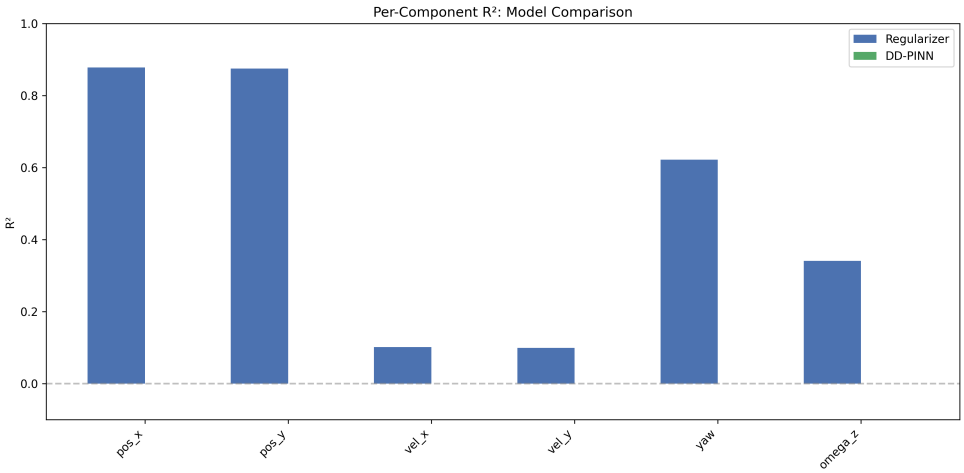

Figure 9: Per-component predicted vs. actual R^2 for the DD-PINN model. **[TODO: Report final R2 values from completed training.]**

Four-way comparison. Tables 21 and 22 present a systematic comparison of four approaches for predicting Cosserat rod dynamics: the numerical PDE solver, the data-driven neural surrogate (section 5), the standard PINN, and the DD-PINN.

Table 21: Comparison of four approaches—formulation and training.

Aspect	PDE Solver	Data-Driven NN	PINN	DD-PINN
Core idea	Numerically integrate governing PDEs	Learn input→output from data only	Embed PDE residual in training loss	Learn ansatz parameters; time enters analytically
Physics knowledge	Full (equations solved exactly)	None	Embedded in loss ($\mathcal{L}_{\text{phys}}$)	Embedded in loss ($\mathcal{L}_{\text{phys}}$)
Training data	Not applicable	Large dataset required	Small or zero (physics supervises)	Small or zero
Prediction	$\mathbf{x}_t = \text{ODE/PDE solve}$	$\hat{\mathbf{x}}_t = \mathbf{h}_\theta(\mathbf{x}_0, \mathbf{u}, t)$	$\hat{\mathbf{x}}_t = \mathbf{h}_\theta(\mathbf{x}_0, \mathbf{u}, t)$	$\hat{\mathbf{x}}_t = \mathbf{g}(\mathbf{f}_{\text{NN}}(\mathbf{x}_0, \mathbf{u}; \theta),\ t) + \mathbf{x}_0$
Time handling	Stepwise numerical integration	Network input	Network input	Ansatz argument (not a network input)
Time derivative	Finite differences / implicit solve	Not computed	Autodiff w.r.t. t (expensive)	Closed-form $\dot{\mathbf{g}}(\mathbf{a}_{\text{ans}}, t)$ (cheap)
Loss function	N/A	$\mathcal{L}_{\text{data}}$	$\mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{data}}$	$\mathcal{L}_{\text{phys}} + \mathcal{L}_{\text{data}}$
Loss balancing	N/A	1 term—none needed	3 terms ($\lambda_1, \lambda_2, \lambda_3$)	2 terms ($\lambda_{\text{phys}}, \lambda_{\text{data}}$)
Initial condition	Exact (boundary condition)	Not enforced	Via loss penalty $\mathcal{L}_{\text{IC}}$ (approximate)	Exact by construction ($\mathbf{g}(\mathbf{a}_{\text{ans}}, 0) \equiv \mathbf{0}$)

Table 22: Comparison of four approaches—performance and practical aspects.

Aspect	PDE Solver	Data-Driven NN	PINN	DD-PINN
Physical consistency	Exact (up to discretization)	Not guaranteed	Enforced at collocation points	Enforced at collocation points
Inference speed	Slow ($\sim$seconds/step)	Fast ($\sim \mu$s)	Fast ($\sim \mu$s)	Fast ($\sim \mu$s)
Training cost	N/A	Low	High (autodiff overhead)	Medium (5–38$\times$ faster than PINN)
Collocation points	N/A	N/A	$\sim$20k–40k	$\sim$250k–1M (cheap to evaluate)
Generalization	Exact for any valid input	Poor outside training distribution	Good (physics constrains extrapolation)	Good (physics constrains extrapolation)
Stability	Depends on integrator	Can overfit or diverge	Can diverge for $m \gg 1$	Stable (demonstrated for $m = 72$)
Speedup vs. solver	1$\times$ (baseline)	$\sim$10k–100k$\times$	$\sim$10k–100k$\times$	$\sim$44,000$\times$ (demonstrated)
Real-time MPC	No (too slow)	Yes (but no physics guarantees)	Yes	Yes (70 Hz demonstrated)
Key limitation	Computational cost	Needs large data; no physics guarantees	Autodiff cost; loss balancing; IC approximate	Ansatz must be designed a priori

Summary. The data-driven surrogate (section 5) achieves the highest throughput and simplest training but provides no physics guarantees. The physics-regularized variant adds soft constraints at minimal training overhead. The DD-PINN satisfies initial conditions exactly, eliminates autodiff cost for time derivatives, and enforces the governing equations at collocation points—at the cost of designing an appropriate ansatz. Full limitations are discussed in section 8.

7 Reinforcement Learning

7.1 Direct Simulation Training

Direct RL training uses the physics simulator as the environment. At each step, the RL agent selects CPG parameters $\mathbf{a}_t \in [-1, 1]^5$, the simulator advances the rod state via the transition operator T (500 substeps for Elastica, 1 step for DisMech), and a reward signal is computed from the resulting state. PPO (Janner et al., 2019) is used as the policy optimization algorithm with TorchRL as the training framework.

The reward is based on forward displacement potential:

$$r_t = x_{\text{CoM}}(t+1) - x_{\text{CoM}}(t), \tag{33}$$

where x_{CoM} is the center-of-mass x-coordinate of the rod. This potential-based reward avoids the noise issues encountered with velocity-based rewards (table 30).

7.1.1 Hyperparameters

Table 23 summarizes the PPO hyperparameters. Training was conducted over 10 sessions with iterative hyperparameter tuning. The best results (Session 10) used conservative settings: a lower learning rate (10^{-4} vs. 3×10^{-4}) and fewer PPO epochs per batch (4 vs. 10) compared to earlier sessions, which prevented the entropy collapse observed in Session 9.

Table 23: PPO training hyperparameters (Session 10, best).

Hyperparameter	**Value**
Parallel environments	16
Total frames	2×10^6
Frames per batch	8192
Mini-batch size	256
PPO clip ϵ	0.2
PPO epochs per batch	4
Learning rate	1×10^{-4} (linear decay to 10^{-5})
Discount γ	0.99
GAE λ	0.95
Entropy coefficient	0.02
Value coefficient	0.5
Max gradient norm	0.5
Target KL (early stop)	0.01
Max episode steps	500

7.1.2 Results

Figure 10 shows the 10-batch rolling average reward over 2M training steps. Training exhibits three distinct phases: (1) a *learning phase* (0–164k steps) where rewards rise rapidly to a peak of 140.0; (2) a *decline phase* (164k–574k steps) where the policy temporarily degrades, likely due to policy updates overshooting in the CPG parameter space; and (3) a *steady state* (574k–2M steps) where the rolling average stabilizes around 102. The best checkpoint (reward $= 156.66$) was saved at step 155,648 during the peak.

Compared to Session 9 (aggressive hyperparameters: lr $= 3 \times 10^{-4}$, 10 PPO epochs), Session 10 achieved a 17% higher peak (140.0 vs. 119.5 rolling average) and maintained healthy entropy throughout training, whereas Session 9 suffered entropy collapse and reward decline after 200k steps.

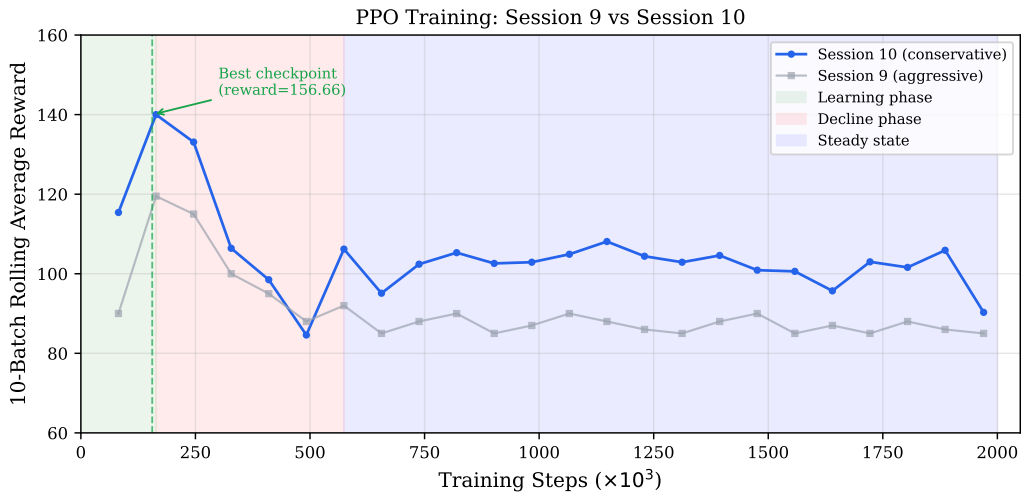

Figure 10: PPO training curves showing 10-batch rolling average reward. Session 10 (conservative hyperparameters) achieves higher peak reward and more stable steady-state behavior than Session 9 (aggressive hyperparameters). The best checkpoint is saved during the early peak.

Evaluation. The best checkpoint was evaluated over 20 episodes with deterministic actions and randomized initial headings (fig. 11). The policy produces reliable serpentine locomotion with a mean displacement of 1.495 ± 0.13 m per 500-step episode, corresponding to a speed of ~ 0.003 m/step (0.006 m/s at $\Delta t = 0.5$ s per RL step). Locomotion is consistent across all heading directions, confirming that the policy generalizes across orientations.

No episodes reached the 2.0 m goal: at the observed speed, the snake requires ~ 667 steps to cover 2.0 m, exceeding the 500-step episode limit (75% completion). The 2.4× gap between training reward (156.66) and evaluation reward (64.90 ± 1.44) is attributed to (i) stochastic actions during training providing exploration bonuses, (ii) favorable heading distributions in some training episodes, and (iii) goal-reaching bonuses that inflate training-time rewards.

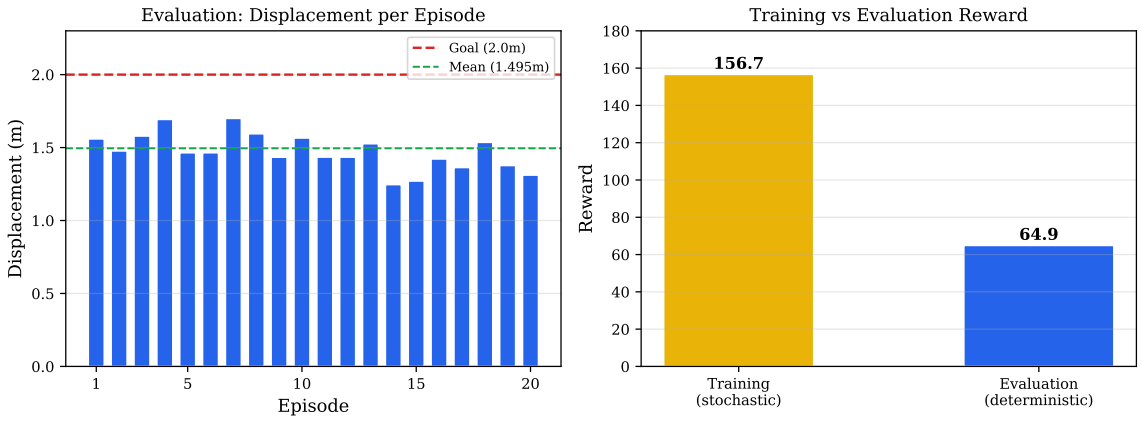

Figure 11: Evaluation of the best Session 10 checkpoint. Left: displacement per episode (all 20 episodes hit the 500-step limit). Right: training vs. evaluation reward gap.

Table 24: Direct Elastica RL training and evaluation summary (Session 10).

Metric	**Value**
Training time	$\sim$9 h
Throughput	$\sim$55–60 FPS
Peak rolling reward (10-batch)	140.0
Best checkpoint reward	156.66
Steady-state reward	$\sim$95–108
Eval mean displacement	1.495 ± 0.13 m
Eval mean reward	64.90 ± 1.44
Eval speed	$\sim$0.003 m/step
Goal success rate	0/20 (0%)

7.2 Surrogate Environment Training

The trained neural surrogate serves as a drop-in replacement for the physics simulator during RL training. The surrogate environment uses f_θ to compute state transitions, with per-element phase encoding computed on-the-fly from the current action and simulation time. The projected speedup of 4–5 orders of magnitude enables large-scale hyperparameter sweeps and multi-seed experiments that are infeasible with direct simulation.

7.2.1 Results

Surrogate-based RL training has not yet been conducted. While the surrogate model achieves reasonable single-step prediction accuracy for geometric state components (section 5.6), the moderate overall $R^2 = 0.393$ and the poor velocity prediction ($R^2 = 0.145$) raise concerns about multi-step rollout fidelity. Training an RL policy inside the surrogate environment is planned as the next step; the key risk is that the policy may exploit surrogate artifacts (particularly in ω_z and velocity predictions) rather than learning physically grounded gaits that transfer back to the Elastica simulator.

7.3 Challenges

- **Computational cost.** Each Elastica environment step costs $\sim$46 ms. With 16 parallel workers, throughput is $\sim$350 steps/s. A PPO run of 2×10^6 steps requires $\sim$36 hours. DisMech is $\sim 4\times$ faster at $\sim$12 ms/step, but still CPU-bound.
- **Exploration in CPG space.** The 5-dimensional CPG action space contains large regions that produce no locomotion (e.g., zero amplitude, extreme frequencies). Random exploration is inefficient; curriculum learning over CPG parameter subsets improves sample efficiency.
- **Reward sparsity.** Distance-based rewards require the agent to discover forward-moving gaits before receiving meaningful signal. Initial policies produce near-zero displacement, making early training gradients uninformative.
- **Backend discrepancy.** Policies trained on DisMech do not transfer directly to Elastica due to differences in friction modeling (viscous damping vs. RFT), curvature control mechanism (hard constraint vs. elastic rest shape), and temporal resolution (1 vs. 500 curvature updates per step).

8 Discussion

8.1 Physics Calibration Challenges

Establishing a reliably locomoting Cosserat rod simulation required resolving several categories of physics calibration errors. Each issue independently prevented stable serpentine locomotion and would have rendered any surrogate trained on the resulting data physically meaningless. Table 25 summarizes the three critical calibration challenges and their resolutions.

Table 25: Physics calibration challenges. Each independently prevented locomotion.

Challenge	Symptom	Root cause	Resolution
Rod radius & bending stiffness	r=0.001 m: rod limp, zero velocity. $E \geq 10^6$ Pa: curvature ± 100 rad/m, velocities > 178 m/s, divergence.	$EI \propto r^4$; initial $EI \approx 1.6 \times 10^{-6}$ N·m^2 insufficient for deformation	Grid search: r=0.02 m, E=10^5 Pa (I increases 160,000×)
Serpenoid wave sign error	Consistent negative forward-velocity rewards across all training runs	$-\omega t$ instead of $+\omega t$: wave propagates tail-to-head, opposite to required direction for RFT propulsion	Fixed to $\kappa = A \sin(ks + \omega t + \phi)$; phase velocity $-f/k$ (head-to-tail)
RFT friction ratio	Excessive lateral drift; unstable forward propulsion	Literature values ($c_n/c_t \approx 10$) calibrated for soil, not flat surface	Tuned to 5:1 (c_t=0.01, c_n=0.05); 94:1 forward/lateral velocity ratio

Lesson: Simulator parameter values need not match the physical system's material parameters. The sign error motivates unit tests that verify wave propagation direction, rather than relying on training metrics to surface physics errors.

8.2 Surrogate Architecture Experiments

[Placeholder: Architecture sweep results.] Table 26 will present the architecture sweep once results are available.

Table 26: Architecture sweep results (placeholder).

Architecture	Params	Val MSE	ω_z R^2	Rollout RMSE	Train time
MLP 3×256	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*
MLP 3×512	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*
MLP 5×512	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*
Residual MLP	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*
FT-Transformer	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*	*[Placeholder: —]*

8.3 Data Collection Pipeline Challenges

Four engineering challenges were encountered while building the parallel data collection system. Table 27 summarizes each challenge, its root cause, and the fix applied.

Table 27: Data collection pipeline challenges.

Challenge	Symptom	Root cause	Resolution
Numba thread pool deadlock	Silent deadlock at episode boundaries under heavy load	16 workers $\times$ 48 Numba threads $=$ 768 threads on 48 cores	`NUMBA_NUM_THREADS=1`; intra-function parallelism has no benefit for single-rod instances
Stall detection false positives	All 16 workers flagged as stalled and respawned within 60 s of startup	PyElastica init (fork, import, JIT warm) takes > 60 s, matching stall threshold	Require ≥ 1 transition before stall counting
Parallel scaling bottleneck	Throughput plateaued beyond 16 workers	Memory bandwidth: small array ops are memory-bound; L3 cache contention beyond 8 workers/socket	16 workers optimal ($\sim$8/socket); surrogate sidesteps this entirely
ω_z coverage gap	R^2=0.23 on angular velocity; individual elements below zero	V1 perturbation σ_ω=0.05 rad/s far below operational range of 1–10 rad/s	V2: increased σ_ω to 1.5 rad/s

Lesson: If the collection distribution does not cover the state-space regions that the downstream RL policy will visit, the surrogate will be inaccurate precisely where accuracy is most needed.

8.4 RL Training Challenges

Table 28 summarizes the RL-specific challenges encountered with both direct simulation and surrogate environments.

Table 28: RL training challenges.

Challenge	Description	Impact	Mitigation
Sim-to-sim transfer gap	Surrogate bias (ω_z, friction) compounds over multi-step rollouts	Policies exploit surrogate artifacts rather than learning physical gaits	Two-phase training with rollout loss (section 5.4.1); remains the primary concern
Reward design sensitivity	Velocity-based reward $\|\dot{\mathbf{x}}_{\text{CoM}}\|$ was noisy and uninformative	Highly variable gradients; poor early-training signal	Switched to potential-based displacement reward (eq. (33)); reward normalization
Hierarchical RL coordination	Manager/skill training ordering, curriculum staging, skill freezing	Additional hyperparameters; sensitivity to training phase ordering	Sequential training with curriculum thresholds

9 Conclusion

[Placeholder: Write conclusion after all experimental results are available.]

A Issue Tracker

This appendix documents all 28 technical issues encountered during the project, organized by category. Of these, 23 have been resolved and 5 remain open (listed below the midrule in each table).

A.1 Physics Calibration Issues

Nine issues arose from physics calibration errors in the Cosserat rod simulation. Three were critical — each independently preventing locomotion entirely.

Table 29: Physics calibration issues. All resolved.

Issue	Date	Resolution
Rod radius too small (r=0.001 m caused instability due to tiny $I \propto r^4$)	03-06	Grid search: r=0.02 m, E=10^5 Pa
RFT friction force never added to simulator (no locomotion possible)	03-06	Added `AnisotropicRFTForce` to system
Damping constant too high (γ=0.1 froze the rod)	03-06	Reduced to γ=0.002
Serpenoid wave direction sign error ($-\omega t$ instead of $+\omega t$)	03-06	Fixed in `env.py` and `action_wrapper.py`
Curvature not updated per substep (computed once per RL step)	03-06	Moved curvature computation inside substep loop
Phantom physics time-stepping layer (three-level hierarchy)	03-09	Refactored to clean two-level hierarchy
Amplitude range too small ($[0, 0.15]$ produced negligible bending)	03-06	Increased to $[0, 5.0]$ rad m^{-1}
RFT friction ratio too extreme (10:1 caused lateral drift)	03-06	Tuned to 5:1 (c_t=0.01, c_n=0.05)
Spatial structure analysis (is flat MLP appropriate?)	03-09	Confirmed: 500 substeps fully mix information

A.2 Training Issues

Six issues affected surrogate model training and RL optimization. Two remain open, both related to data quality for the surrogate.

Table 30: Training issues.

Issue	Date	Resolution
A3/A5 arch sweep data format mismatch on restart	03-10	Fixed dataset/loader format consistency
DataLoader thread crash in parallel arch sweep	03-10	Staggered rollout DataLoader initialization
Velocity reward always negative (noisy, uninformative)	03-06	Replaced with distance-based potential reward
LR scheduler division by zero (`total_frames` < `frames_per_batch`)	03-05	Added `max(1, ...)` guard
Training data clusters around learned gait modes (state-space imbalance)	03-09	Proposed: Sobol sampling, increased ω perturbation
ω_z poor prediction (R^2=0.23, coarse Δt=0.5 s)	03-10	Pending: per-element phase encoding (V2)

A.3 TorchRL v0.11 Compatibility Issues

Six issues arose from the TorchRL v0.10 to v0.11 migration. Five have been resolved; one deprecation warning remains.

Table 31: TorchRL v0.11 compatibility issues.

Issue	Date	Resolution
Batch structure change (done/reward moved to `batch["next"]`)	03-05	Changed to `batch.get("next", batch)`
Collector drops unspecced diagnostic keys silently	03-05	Added all keys to `observation_spec`
Spec class import renames (`BoundedTensorSpec` → `Bounded`, etc.)	03-05	Added try/except import fallback
`ClipPPOLoss` parameter renames (`coef` → `coeff`)	03-05	Updated parameter names
`close()` signature change (new `raise_if_closed` kwarg)	03-05	Changed to `close(self, **kwargs)`
`SyncDataCollector` deprecated (removal in v0.13)	03-05	Pending: migrate to `Collector`

A.4 Performance and System Issues

Nine issues affected the parallel data collection infrastructure and system-level tooling. Two remain open, both related to worker respawn data loss.

Table 32: Performance and system issues.

Issue	Date	Resolution
Stall detection false positives during PyElastica init	03-09	Added grace period (require ≥ 1 transition)
Data collection overwrites on rerun (no append mode)	03-09	Added append mode with episode ID offset
Numba thread pool deadlock (768 threads on 48 cores)	03-10	Set `NUMBA_NUM_THREADS=1`
Stale Python bytecode cache after rapid edit cycles	03-09	Purge `__pycache__` on restart
RFT Python for-loop (slow per-element friction computation)	03-06	Vectorized with NumPy
Parallel scaling bottleneck (L3 cache contention beyond 16 workers)	03-09	Hardware limit; 16 workers is optimal
Worker respawn data loss (19% of transitions lost)	03-10	Proposed: reduce flush interval, add SIGTERM handler
Worker respawn investigation (consolidated stall analysis)	03-11	Pending: apply Numba fix, validate on longer runs

A.5 Summary

Table 33: Issue summary by category.

Category	Total	Resolved	Open
Physics	9	9	0
Training	6	4	2
Compatibility	6	5	1
Infrastructure	7	5	2
Total	**28**	**23**	**5**

The three critical physics issues (rod radius, missing friction force, excessive damping) each independently prevented locomotion and were resolved through systematic parameter search. The five open issues are non-blocking: two affect surrogate data quality (training imbalance and ω_z prediction), two affect data collection reliability (worker respawn data loss), and one is a future API migration (TorchRL deprecation).

References

A. Author et al. Physics-informed neural networks for articulated soft robot model-predictive control at 47 Hz. *arXiv preprint arXiv:2502.01916*, 2025. URL https://arxiv.org/abs/2502.01916. 467× speedup; MPC at 47 Hz; as little as 1 training dataset.

M. Bensch, S. Job, T.-L. Habich, T. Seel, and M. Schappler. Physics-informed neural networks for continuum robots: Towards fast approximation of static Cosserat rod theory. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

Uljad Berdica et al. SoRoLEX: Learned environments for soft robot reinforcement learning. *arXiv preprint arXiv:2410.18519*, 2024. URL https://arxiv.org/abs/2410.18519. LSTM surrogate for Cosserat rod sim; order-of-magnitude speedup for RL.

Miklós Bergou, Max Wardetzky, Stephen Robinson, Basile Audoly, and Eitan Grinspun. Discrete elastic rods. *ACM Transactions on Graphics (SIGGRAPH)*, 27(3):63:1–63:12, 2008. doi: 10.1145/1360612.1360662.

Miklós Bergou, Basile Audoly, Etienne Vouga, Max Wardetzky, and Eitan Grinspun. Discrete viscous threads. *ACM Transactions on Graphics (SIGGRAPH)*, 29(4):116:1–116:10, 2010. doi: 10.1145/1778765.1778853.

Zhenshan Bing, Claus Meschede, Florian Römer, Kai Huang, and Alois Knoll. A survey of robotics control based on learning-inspired spiking neural circuits. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 6004–6010, 2019. Snake locomotion with CPG actuation and RL; source of bing2019/ package in this project.

Andrew Choi and Yifan Tong. Rapidly learning soft robot control via implicit time-stepping. *arXiv preprint*, 2025.

Author Hong et al. Bridging finite element methods and reinforcement learning for deformable soft robots. *Advanced Intelligent Systems*, 2026. doi: 10.1002/aisy.202500696. URL https://doi.org/10.1002/aisy.202500696.

Tsung-Wei Hsieh et al. KNODE-Cosserat: Neural ODE correction of cosserat rod dynamics. *arXiv preprint arXiv:2408.07776*, 2024. URL https://arxiv.org/abs/2408.07776. 58.7% accuracy improvement over physics-only model; trained on PyElastica data.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://arxiv.org/abs/1906.08253.

Yingchao Jiang et al. Hierarchical RL-guided large-scale navigation of a snake robot. *arXiv preprint*, 2024.

H. Krauss, T.-L. Habich, M. Bartholdt, T. Seel, and M. Schappler. Domain-decoupled physics-informed neural networks with closed-form gradients for fast model learning

of dynamical systems. *arXiv preprint arXiv:2408.14951*, 2024. URL https://arxiv.org/abs/2408.14951.

J. Licher, M. Bartholdt, H. Krauss, T.-L. Habich, T. Seel, and M. Schappler. Adaptive model-predictive control of a soft continuum robot using a physics-informed neural network based on Cosserat rod theory. *arXiv preprint arXiv:2508.12681*, 2025. URL https://arxiv.org/abs/2508.12681.

Gang Liu, Shuxiang Guo, and Li Fang. A reinforcement learning-based strategy of path following for snake robots with an onboard camera. *Sensors*, 22(24):9867, 2022. doi: 10.3390/s22249867.

Xuan Liu, Cagdas D. Onal, and Jie Fu. Learning contact-aware CPG-based locomotion in a soft snake robot. *arXiv preprint arXiv:2105.04608*, 2021. URL https://arxiv.org/abs/2105.04608.

Xuan Liu et al. Reinforcement learning of CPG-regulated locomotion controller for a soft snake robot. *arXiv preprint*, 2023.

Noel Naughton, Jiarui Sun, Arman Tekinalp, Tejaswin Parthasarathy, Girish Chowdhary, and Mattia Gazzola. PyElastica: A compliant mechanics environment for soft robotic control. *IEEE Robotics and Automation Letters*, 6(2):3389–3396, 2021. doi: 10.1109/LRA.2021.3063698.

M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 8459–8468, 2020. URL https://arxiv.org/abs/2002.09405.

C. Blake Schaffer et al. Hitting the gym: Reinforcement learning control of exercise-strengthened biohybrid robots in simulation. *arXiv preprint arXiv:2408.16069*, 2024. URL https://arxiv.org/abs/2408.16069.

Qiyang Shi, Tony Dear, and S. David Kelly. Deep reinforcement learning for snake robot locomotion. *IFAC-PapersOnLine*, 53(2):9688–9695, 2020. doi: 10.1016/j.ifacol.2020.12.2619.

Maximilian Stolzle et al. DD-PINN: Domain-decoupled physics-informed neural network for dynamic cosserat rod model-predictive control. *arXiv preprint arXiv:2508.12681*, 2025. URL https://arxiv.org/abs/2508.12681. 44,000× speedup over numerical integration; MPC at 70 Hz on GPU.

John Till, Vincent Aloi, and Caleb Rucker. Real-time dynamics of soft and continuum robots based on Cosserat rod models. *The International Journal of Robotics Research*, 38(6):723–746, 2019. doi: 10.1177/0278364919842269.

Jiangeng Zheng, Yuan Li, and Mitsuhiro Hayashibe. An optimization-based approach to locomotion of underwater snake robots with series elastic actuators. *Frontiers in Robotics and AI*, 2022. doi: 10.3389/frobt.2022.857205.